Recruiting and Application Management System

Version 2.5
Five Guys
Team 3
2026/10/28

Change Logs

The main features the team produced in each iteration are listed below.

Iteration 1

- Project introduction, scope and assumptions.
- Functional and non-functional requirements.
- Fifteen use cases (three per member) with use case diagrams for the Applicant, Recruiter and Administrator.
- Sequence diagrams for the fifteen use cases.
- Domain model.
- Jira board, GitHub repository and team website set up.

Iteration 2

- After the Iteration 1 review we rewrote the Introduction and the business assumptions. Interviews and offer negotiation now happen outside the system. The system only records the stage.
- Revised the domain model. UserAccount is now the superclass of Applicant, Recruiter and Administrator. StageRecord (stage, date, changedBy) holds the application history. The direct Application–Resume link is removed. Since Release v2.5 the code follows the model: UserAccount is an abstract superclass of the three role classes.
- Updated the sequence diagrams to match the revised domain model and added the design class diagram (Section 7).
- Implemented the back end (Spring Boot, Java 21) for all fourteen use cases: Search Available Jobs, Register Account, User Login, Update Applicant Profile, Submit Job Application, Save Job Posting, Track Application Stage, Withdraw an Application, Create and Publish a Job Posting, Manage Job Posting, Review Submitted Applications, Update Application Stage, Extend Job Offer and Manage Accounts and Roles. The sample jobs come from jobs.csv.
- Built the React front end (frontend folder) and connected it to the back end for all three roles. It covers sign in, job search, registration, applications, withdrawal, profile, postings, candidate review with resume download, and account management. Two plain HTML pages (Submit Application, My Applications) remain for a quick demo.
- Added the Implementation Overview, Testing Plan with thirty-four test cases, Setup / Installation Guide and User Manual (Sections 8–12).
- Removed the use case Respond to Job Offer (4.2.1.9). Its sequence diagram (6.9 in Iteration 1), its bubble in the applicant use case diagram and requirement 3.1.4 are gone too. Accepting or declining an offer happens outside the system. The recruiter records the result with Update Application Stage (4.2.2.4). Schedule an Interview became Extend Job Offer (4.2.2.5).
- We released the code on GitHub in steps. v1.0 is the Iteration 1 back end. v2.0 to v2.5 are Iteration 2: v2.4 connected the front end, v2.5 added User Login and the UserAccount inheritance; v2.5.1 and v2.5.2 are clean-ups that connect the job search and offer pages to their endpoints. Every step was merged through a pull request.

Addressing Previous Feedback

The table lists Dr. Ren's comments on Iteration 1 and the rubric items where we lost points. The last column says what we changed. Rows 7 to 10 cover the comments on the Iteration 2 submission (October 26, 2026). Items with full marks are not repeated.

Team-level feedback

#	Instructor comment (Iteration 1)	What we changed	Where
1	The introduction is too general and contains course-related content; the document targets the customer and the development team. (Intro + Project Overview 0.6/0.75)	We rewrote Sections 1.1 to 1.5 for the client and the development team. They now cover the purpose, the business problem, the proposed system, the business goals and the readers. Course-related text is removed.	Section 1
2	2.2 Assumptions should be business level rather than technical requirements; the first paragraph can be deleted.	2.2 now lists seven business assumptions. We deleted the technical paragraph.	Section 2.2
3	3.1.7: make interview scheduling and offer negotiation an offline process managed by the company; the system only records the status of the application. (Requirements 0.8/1)	We reworded 3.1.4 and 3.1.7. Interviews and offer negotiation happen outside the system. The system records only the stage changes, with date and user. Schedule an Interview was removed and replaced. Track Application Stage step 6 now reports the stage history.	Sections 3.1.4, 3.1.7, 4.1.2, 4.2.2.5
4	Use case diagram: Respond to Job Offer is not interpreted in the functional requirements.	Respond to Job Offer was removed from the use case diagram, the use cases (4.2.1.9) and the sequence diagrams (6.9). Accepting or declining an offer is part of the negotiation outside the system. The recruiter records the result as ACCEPTED or DECLINED with Update Application Stage (4.2.2.4).	Sections 3.1.4, 3.1.7, 4.1.1, 4.2.2.4
5	Domain model: an application includes a resume through an applicant, not directly; Administrator, Applicant and Recruiter share attributes and belong under an abstract UserAccount; the UserAccount–Recruiter multiplicity must reflect one role per account; the Application–Stage composition must clearly be an application-specific history record. (Domain Model 1.5/2)	Direct Application–Resume association removed. The resume is reached through the applicant. UserAccount is now an abstract superclass of Applicant, Recruiter and Administrator holding the shared attributes (email, name, status). The UserAccount–Recruiter association is replaced by generalization, so each account is exactly one Applicant, Recruiter or Administrator. Stage is now StageRecord (stage, date, changedBy). Each Application owns its StageRecords as its history. Table 1 and the text under Figure 2 explain this.	Section 5, Figure 2, Table 1
6	Document quality and consistency (0.4/0.5, no specific comment).	We checked the terms (stage vs. status, posting vs. job). We renumbered the figures and tables. We rebuilt the table of contents and added the Change Logs.	Whole document
7	Great team website and release management! (Iteration 2)	Kept the practice. Every change goes through a reviewed pull request. Releases v1.0 to v2.5 are on GitHub. The website has the Iteration 2 files.	GitHub, website

#	Instructor comment (Iteration 1)	What we changed	Where
8	Make sure all texts in your system are shown in English.	Checked the React front end, the two back-end pages, every error message and the sample data. All text is in English. A search for Chinese or other non-English text in frontend/src and backend/src/main finds none.	frontend, backend
9	Lack communication with the persistent layer (database) in several SDs. It's better to include database repository in the diagram to address the actual data CRUD activities.	Sequence diagrams 6.1, 6.9 and 6.14 now have a Repository lifeline. It shows the real calls: findByStatus, findById, findByEmailIgnoreCase, countByRoleAndStatus, save, delete, saveAndFlush. The other sequence diagrams get the same lifeline in Iteration 3.	Section 6
10	It would be better to integrate all class diagrams into a complete system-level design in the final deliverable.	One system-level design class diagram (Figure 3) replaces the per-member diagrams. It uses the domain model as the frame. It adds every controller (except DemoIntegrationController and SystemInfoController), service, entity and repository on the main branch, with their dependencies. Table 3 groups the same classes into subsystems.	Section 7, 8.3

Individual use case feedback

Member	Use case	Instructor comment	What we changed	Where
Luyu Niu	4.2.1.1 Search Available Jobs	Searching a job does not require that a job already exists; the second precondition is unnecessary.	Precondition 2 deleted.	Section 4.2.1.1
Luyu Niu	4.2.2.1 Create and Publish a Job Posting	The use case contains another use case (save as draft). Posting from a saved draft could be an alternative scenario or a separate use case.	The main flow now creates and publishes a posting from scratch. Saving a draft and resuming it later is extension 5a.	Section 4.2.2.1
Xinyi Chen	4.2.1.4 Update Applicant Profile	4.2.1.4: According to the main flow, better to rename it as "Update Applicant Profile"	Renamed the use case to Update Applicant Profile in the use case table, the Applicant use case diagram and sequence diagram 6.4. Main flow step 1 now reads "view and update".	Sections 4.1.1, 4.2.1.4, 6.4
Yuting Mei	4.2.1.5 Submit Job Application	An application includes a resume through an applicant. They don't directly have the relationship.	Removed the direct Application–Resume association from the domain model. The resume now belongs to the Applicant. The sequence diagram now checks the application at submission and rejects a duplicate application. It keeps a copy of the submitted materials and shows the success and error paths.	Sections 4.2.1.5, 6.5

Member	Use case	Instructor comment	What we changed	Where
Yuting Mei	4.2.2.2 Manage Job Posting	The sequence diagram should clearly show the validation and update process, including authorization checks and job posting status restrictions.	Revised the use case and sequence diagram to align with the implementation. Validation now occurs after the recruiter submits the changes. The updated sequence diagram includes company ownership verification, job posting status checks, and the update operation.	Sections 4.2.2.2, 6.10
Spencer Hammack	4.2.1.7 Track Application Stage	Interview scheduling and offer negotiation should be handled offline. The system should only record application statuses.	Replaced the interview information with the application stage history in the use case. The sequence diagram now returns the current stage and the stage history.	Sections 4.2.1.7 and 6.7
Spencer Hammack	4.2.2.5 Extend Job Offer (Formerly Schedule an Interview)	Interview scheduling and offer negotiation should be handled offline. The system should only record application statuses.	Replaced the use case Schedule an Interview with Extend Job Offer. The revised use case and sequence diagram record the OFFER stage after the recruiter confirms the offer. Offer communication and negotiation stay outside the system.	Sections 4.2.2.5 and 6.13
Zhengshi (Peter) Chen	4.2.2.4 Update Application Stage	Interview scheduling and offer negotiation should be handled offline. The system should only record application statuses.	Replaced Respond to Job Offer with Update Application Stage, which the recruiter performs. The recruiter records the applicant's decision after talking with them outside the system.	Sections 4.2.2.4, 6.12
Zhengshi (Peter) Chen	4.2.1.9 Respond to Job Offer / 4.2.2.4 Update Application Stage	Respond to Job Offer is not properly reflected in the functional requirements.	Updated the functional requirements and the API endpoints. Removed the offer response feature. Added domain, service, controller and integration tests for the new flow.	Sections 3.1.4, 4.2.2.4; Tables 4–6

Table of Contents

1. Introduction 7
2. Project Overview 8
3. Requirements 9
4. Use Case Model 11
5. Domain Model (Analysis) 26
6. Sequence Diagrams (Design) 28
7. Design Class Diagram (DCD) 42
8. Implementation Overview 44
9. Testing 48
10. Team Contribution 53
11. Setup / Installation Guide 54
12. User Manual 55

1. Introduction

1.1 Purpose

This document specifies the Recruiting and Application Management System. It states what the system must do, who uses it, and how it is designed. It covers the requirements, the use cases, and the analysis and design models. It also has guides for installing and using the system. The client and the team use it to agree on the scope and to check each release.

1.2 Business Problem

Many organizations still run hiring with email and spreadsheets. Job postings are published on one site. Resumes arrive by email. The stage of each candidate is kept in a spreadsheet or in one recruiter's memory. As a result, applicants cannot see where their applications stand, so they email recruiters, who answer by hand. Recruiters cannot see all applications for a posting in one place, so some candidates are missed and others are reviewed twice. Later, nobody can tell who changed a stage or when.

1.3 Proposed System and Its Users

The Recruiting and Application Management System is a web application that keeps the hiring record in one place. It serves three kinds of users.

- Applicants keep a profile and resumes, search open job postings, apply, and follow the stage of each application.
- Recruiters work for a hiring company. They publish job postings, review the applications they receive, and record the stage of each application.
- Administrators operate the system. They manage user accounts and give each account its role.

The system only records the hiring process. The hiring company runs interviews and offer negotiation outside the system. The recruiter then records the result as a new stage, such as INTERVIEW or OFFER, and the applicant sees it. The applicant answers an offer outside the system, and the recruiter records the answer. Each stage change is saved with who made it and when.

1.4 Business Goals

The client should get these results.

- Applicants can check the current stage of every application at any time without contacting a recruiter.
- Recruiters see all applications for their company's job postings in one list, each with its current stage.
- Every stage change can be traced to a person and a time.
- Job seekers find open postings by keyword, location and employment type, and expired postings never appear.
- Applicant data is visible only to the applicant and to the recruiters who handle the application.

1.5 Intended Readers and Document Organization

This document has two groups of readers. The client reads Sections 2 to 4 to confirm the scope, the requirements and the use cases. The development team reads every section and works from Sections 5 to 9 when it builds and tests the system.

Section 2 says what is inside and outside the system and lists the business assumptions. Section 3 states the functional and non-functional requirements. Section 4 gives the use case model. Sections 5 to 7 give the domain model, the sequence diagrams and the design class diagram. Sections 8 and 9 describe the implementation and the tests. Section 10 records the contribution of each team member. Sections 11 and 12 explain how to install and how to use the system.

2. Project Overview

The system supports the recruiting process from a job posting to a final outcome. Applicants search and apply, recruiters post jobs and move applications through the stages, and administrators manage accounts.

2.1 Product Perspective

The system is a standalone web application. It is not part of a larger product and nothing else depends on it. Users open it in a web browser. The system keeps its own database and stores the resume files that applicants upload.

The system holds all records of the recruiting process. This covers accounts and roles, applicant profiles, resumes, postings, and applications with their stage history.

The system does not run interviews or offer negotiation. It only records the result as a stage. It does not post jobs to outside job boards, and it sends no email to applicants.

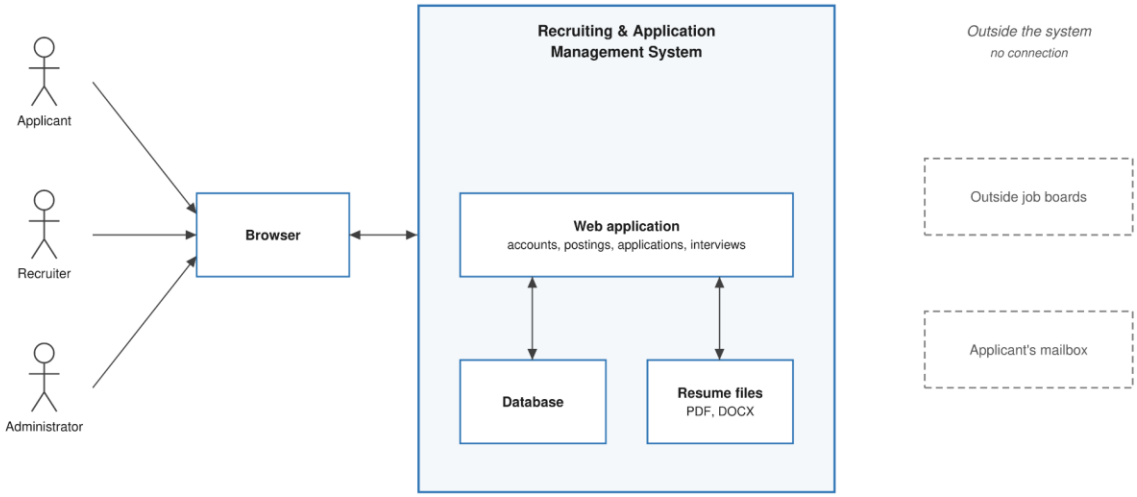


Figure 1. System context

2.2 Assumptions and Dependencies

We built the system on these business assumptions. Each one simplifies the problem.

- All hiring companies follow one shared hiring process. Every application moves through the same set of stages.

- Each recruiter works for exactly one company, and each job posting belongs to exactly one company. A recruiter handles only the postings and applications of that company.
- Each user account has exactly one role: applicant, recruiter or administrator.
- Interviews and offer negotiation are carried out by the hiring company outside the system. The system only records their outcome as a stage of the application.
- Communication with applicants, such as interview invitations and offer letters, takes place outside the system. The system sends no notifications. Applicants learn about a change by checking their applications.
- Applicants are responsible for the accuracy of their profiles and resumes. The system stores each resume as a file and does not evaluate it. Every hiring decision is made by a person.
- The organization that operates the system appoints the first administrator when the system goes live.

Some assumptions would be expensive to change. Automatic notices would need a message at every stage change. A hiring process per company would need stages and access rules per company. Interview scheduling would add interview times and places to the data.

3. Requirements

3.1 Functional Requirements

3.1.1 User Account and Access

New applicants and recruiters can create an account. During registration, users choose Applicant or Recruiter as their role. Recruiters also provide their company information. Each account must have a unique email address. The system rejects a registration if the email address is already in use. Registered users can log in and log out.

3.1.2 Applicant Profile and Application Materials Maintenance

Applicants can update their profile and upload resumes.

3.1.3 Job Search and Application Submission

Anyone can browse, search and filter job postings without logging in. The filters are title keyword, location and employment type. Each posting shows its title, company, location, employment type and closing date. After logging in, applicants can save postings and view them later. An applicant must be logged in to apply. The posting must be open. The applicant must provide the required information and materials. The same applicant cannot apply twice to the same posting. A submitted application enters the process at the SUBMITTED stage.

3.1.4 Application Tracking and Applicant Actions

Applicants can view their submitted applications and the current stage of each one. An application can be withdrawn while it is SUBMITTED, UNDER_REVIEW or INTERVIEW. A withdrawn application leaves the active process. Applicants can also see the recorded outcomes of their own applications.

Accepting or declining an offer happens outside the system. The recruiter records the decision as ACCEPTED or DECLINED (Update Application Stage).

3.1.5 Job Posting Management

Recruiters can create and publish job postings for their company. They can save drafts. They can edit or close their company's postings. Draft postings are not visible to applicants. The system keeps the record of a closed posting, but it no longer appears in searches or accepts applications. Applications submitted before the closing continue through the process.

3.1.6 Application Review and Recruiting Process

Recruiters can review submitted applications. They see the applicant's profile, resume and other materials. They can move each application through the process. The main path is SUBMITTED → UNDER_REVIEW → INTERVIEW (optional) → OFFER → ACCEPTED / DECLINED. An application can be REJECTED at any stage before OFFER. Before OFFER, the applicant can also withdraw it (WITHDRAWN).

3.1.7 Recruiting Outcome

Interviews and offer negotiation take place outside the system and are managed by the company. Recruiters record the outcome as a stage. The stages they record are INTERVIEW, OFFER, REJECTED, ACCEPTED and DECLINED. The applicant sets WITHDRAWN. The system records each stage change with its date and the user who made it.

3.1.8 System Administration

Administrators can view user accounts, activate or deactivate accounts, and assign or update user roles. The role decides what an account can do.

3.2 Non-functional Requirements

3.2.1 Usability

- The screens for all three roles should use the same terms and the same menu layout.
- Error messages should say what went wrong and what the user must fix.
- The system should use the same stage names everywhere.

3.2.2 Performance

- Common operations such as job searching, viewing applications, and updating application stages should complete within 3 seconds.

3.2.3 Reliability

- Successfully saved applications, job postings, and application stage records should not be lost after a user logs out.
- Failed or invalid operations should not be recorded as successful and should not change existing data.
- Temporary system failures should not cause previously saved data to be lost.

3.2.4 Security and Privacy

- Users must log in. Each user can use only the functions and data their role allows.
- Unauthorized users must not see or change applicant profiles, resumes or application records.
- Passwords should not be stored in plain text.

3.2.5 Compatibility

- The system must work in current browsers such as Chrome, Edge, Firefox and Safari. It must not need browser plug-ins or desktop programs.

3.2.6 Testability and Maintainability

- Tests should cover the main recruiting process and the main business rules.
- Requirements, use cases, and tests should use consistent terminology.

3.3 Requirements Clarifications

Q1: Can users browse, search, and filter jobs without logging in?

A1: Yes. Anyone can browse, search and filter job postings without logging in, but they must log in to apply.

Q2: Is the interview stage required for every application?

A2: No. The interview stage is optional and may be skipped when recruiters think it is unnecessary.

Q3: What information is used to identify duplicate accounts?

A3: Email address.

Q4: Who can create an account by self-registering?

A4: Applicants and recruiters.

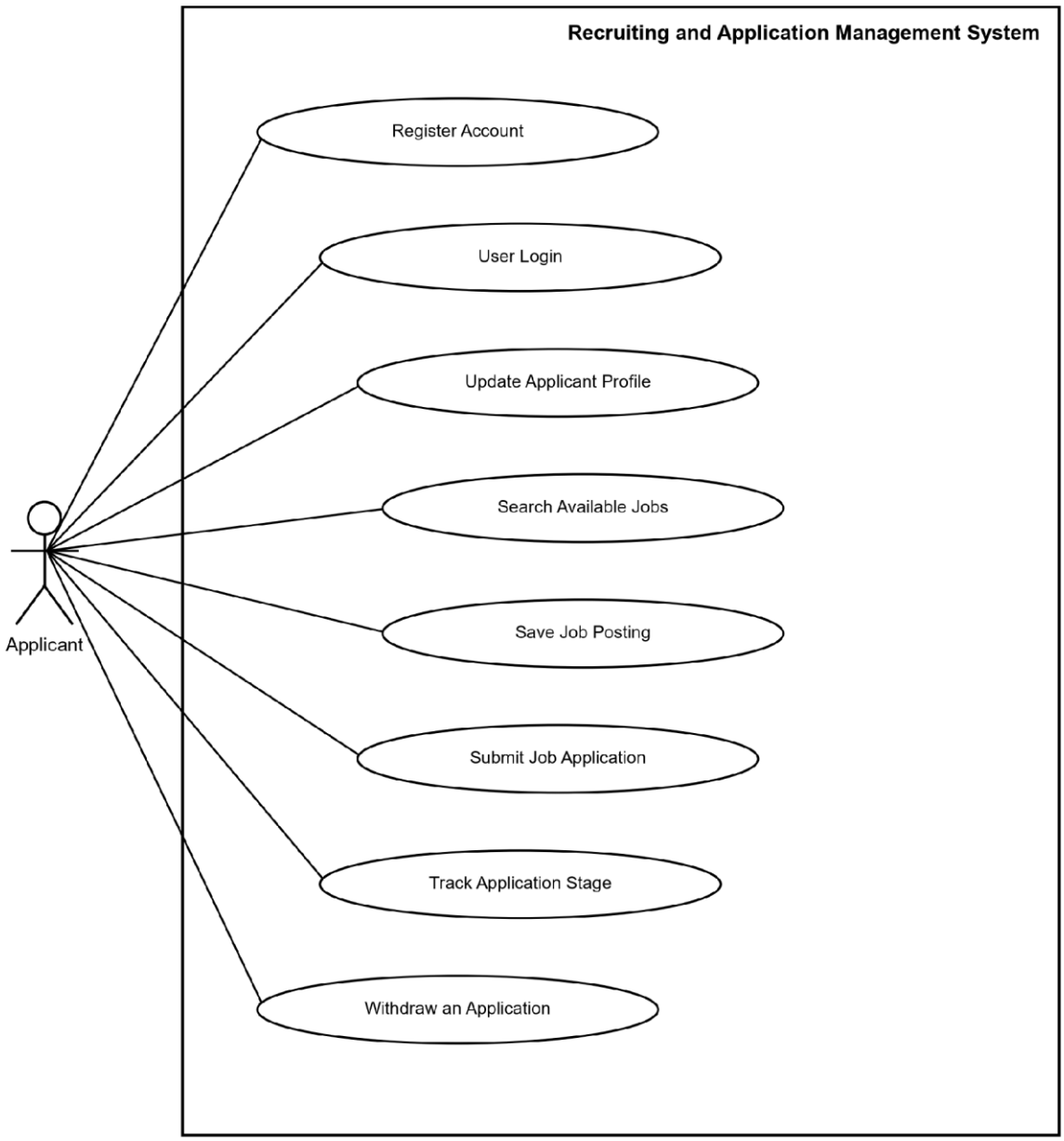
Q5: How is a recruiter associated with a company?

A5: Recruiters provide the company information during registration.

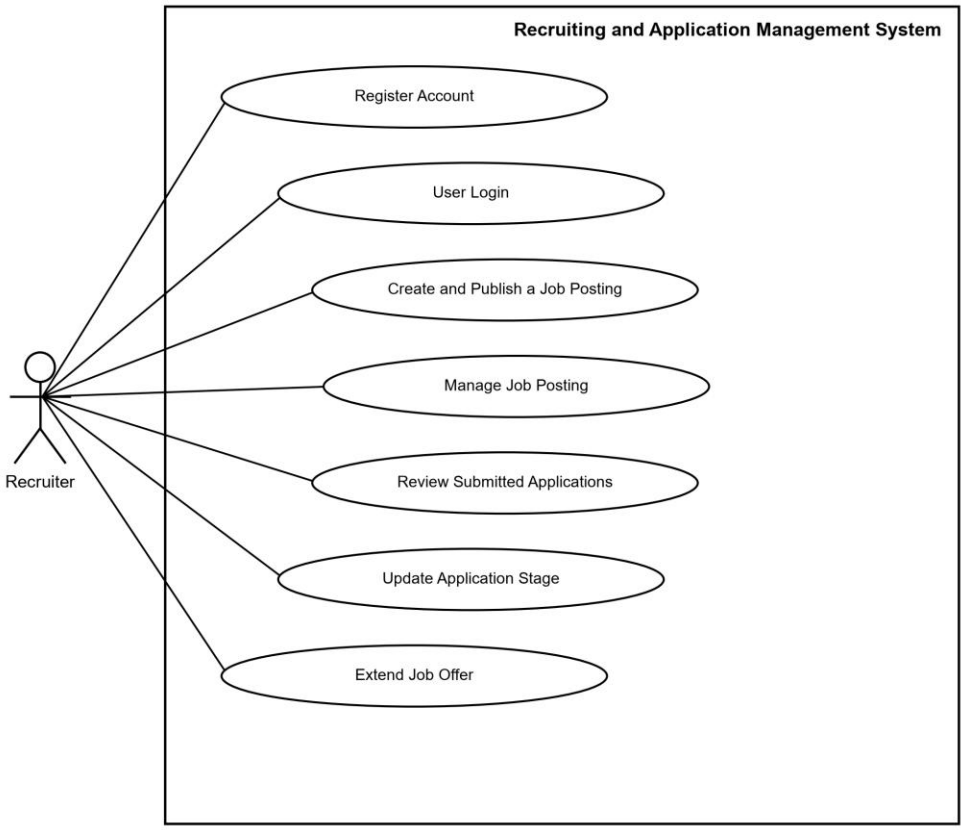
4. Use Case Model

4.1 Use Case Diagram

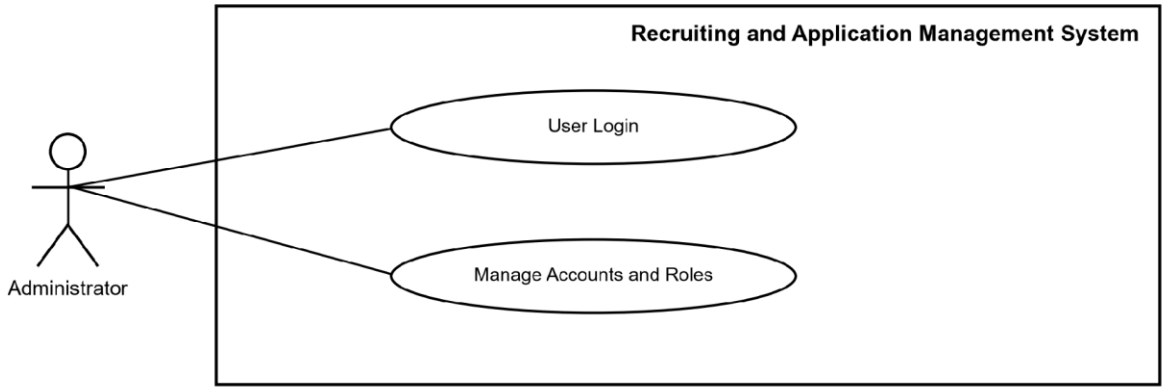
4.1.1 Selected Actor: Applicant



4.1.2 Selected Actor: Recruiter



4.1.3 Selected Actor: Administrator



4.2 Fully-Dressed Use Cases

4.2.1 Actor Name: Applicant

4.2.1.1 Use Case Name: Search Available Jobs

Use Case Name	**Search Available Jobs**
Author	**Luyu Niu**
Actor	**Applicant**
Preconditions	1. The recruiting system is online and operational.
Postconditions	1. A list of jobs satisfying the applicant's criteria is returned. 2. No application is submitted.
Main Flow	1. The applicant starts a job search. 2. The system asks for the search criteria. 3. The applicant provides a job title keyword, other criteria, or both. 4. The system validates the criteria. 5. The system selects the postings that are open and match every criterion. 6. The system returns the matching postings, most recently posted first. 7. For each posting the system reports the title, company, location, employment type and closing date.
Extensions	3a. The applicant provides no criteria. 3a1. The system returns all open postings, most recently posted first. 4a. The criteria are not valid. 4a1. The system reports the problem and asks for corrected criteria. 5a. No posting matches the criteria. 5a1. The system reports that no posting matches and keeps the criteria. 6a. The applicant changes the criteria. 6a1. The system repeats steps 4 to 7 with the new criteria.
Special Requirements	● The system returns search results within 3 seconds for 95% of searches when up to 1,000 postings are published. ● A posting that was closed before the search runs is not returned.

4.2.1.2 Use Case Name: Register Account

Use Case Name	**Register Account**
Author	**Xinyi Chen**
Actor	**Applicant, Recruiter**

Preconditions	1. The user is not currently logged into the system.
Postconditions	1. A new user account exists in the system. 2. The account is assigned the chosen role. 3. If the user registers as a Recruiter, the recruiter is associated with the provided company.
Main Flow	1. The user starts account registration. 2. The system asks for the required registration information. 3. The user chooses Applicant or Recruiter as the role and provides the required information. 4. The system validates the provided information. 5. The system checks whether the email address is already registered and, if not, creates the account with the chosen role. 6. The system confirms that the registration is completed successfully.
Extensions	3a. The user chooses Recruiter as the role. 3a1. The system asks for company information. 3a2. The user provides the required company information. 3a3. The use case continues at Step 4. 4a. The required information is invalid or missing. 4a1. The system reports that the information is invalid or missing and asks for corrected information. 5a. The email address is already registered. 5a1. The system reports that the email address is already used. The user enters another email address or cancels the registration.
Special Requirements	● User credentials and personal information should be protected from unauthorized access. ● Passwords must not be stored in plain text.

4.2.1.3 Use Case Name: User Login

Use Case Name	**User Login**
Author	**Xinyi Chen**
Actor	**Applicant, Recruiter, Administrator**
Preconditions	1. The user has a registered account.
Postconditions	1. The user is authenticated by the system. 2. The user has access to the functions permitted by the account's role.

Main Flow	1. The user starts the login process. 2. The system asks for the registered account login information and password. 3. The user provides the login information and password. 4. The system validates the provided login information. 5. The system authenticates the user by checking the credentials and account status. 6. The system provides access to functions permitted by the user's role and authorization.
Extensions	5a. The login information or password is incorrect. 5a1. The system reports that the login information or password is incorrect and asks the user to try again. 5b. The account is deactivated. 5b1. The system refuses login and reports that the account is inactive.
Special Requirements	● Authentication information must be protected from unauthorized access.

4.2.1.4 Use Case Name: Update Applicant Profile

Use Case Name	**Update Applicant Profile**
Author	**Xinyi Chen**
Actor	**Applicant**
Preconditions	1. The applicant is identified by the system.
Postconditions	1. The applicant profile is saved with the updated information.
Main Flow	1. The applicant requests to view and update their profile. 2. The system provides the applicant's current profile. 3. The applicant provides the updated required profile information. 4. The system validates the provided information. 5. The system saves the profile information. 6. The system confirms that the profile is saved successfully.
Extensions	3a. The applicant decides not to save the changes. 3a1. The existing profile information remains unchanged. 4a. The required information is invalid or missing. 4a1. The system reports that the information is invalid or missing and asks for corrected information.

Special Requirements	• Applicant profile information must not be visible or modified by unauthorized users.

4.2.1.5 Use Case Name: Submit Job Application

Use Case Name	**Submit Job Application**
Author	**Yuting Mei**
Actor	**Applicant**
Preconditions	1. The applicant is identified by the system.
Postconditions	1. A new application is created and stored in the system. 2. The application is associated with the applicant and the chosen job posting. 3. The application stage is set to *SUBMITTED*.
Main Flow	1. The applicant requests to apply for a chosen job posting. 2. The system provides the application requirements and retrieves the applicant's profile and available application materials. 3. The applicant chooses an existing resume or provides a new one, and any other required materials. 4. The system validates the application information. It also verifies that the job posting is still open and the applicant has not already applied. 5. The system presents the completed application for confirmation. 6. The applicant confirms submission. 7. The system creates and stores the application. 8. The system records the application stage as *SUBMITTED*. 9. The system confirms successful submission.
Extensions	2a. Applicant profile is incomplete. 2a1. The system identifies the missing required profile information. 2a2. The system asks the applicant to complete the missing profile information. 2a3. The applicant completes the missing information, and the use case continues at step 4. 3a. A required application material is missing. 3a1. The system identifies the missing required material. 3a2. The system asks the applicant for the missing material. 3a3. The applicant provides the missing material, and the use case continues at step 4. 4a. The job posting is no longer open for applications. / 4a1. The system reports that the job posting has closed. 4a2. The system does not create the application. 4a3. The use case ends.

	4b. The applicant has already submitted an application for the chosen job posting. 4b1. The system informs the applicant that an application has already been submitted for this job posting. 4b2. No duplicate application is created, and the use case ends. 6a. The applicant cancels the submission. 6a1. The system does not create or submit the application. 6a2. The use case ends.
Special Requirements	● Only the applicant who owns the account may submit an application using that applicant's profile and application materials. ● An applicant may submit only one application for the same job posting. ● A submitted application keeps its own copy of the materials. Later changes to the applicant's profile or materials do not change it. ● Applicant data must be protected from unauthorized access ● The resume is a PDF or DOCX file of at most 5 MB.

4.2.1.6 Use Case Name: Save Job Posting

Use Case Name	**Save Job Posting**
Author	**Yuting Mei**
Actor	**Applicant**
Preconditions	1. The applicant is identified by the system.
Postconditions	1. The chosen job posting is saved to the applicant's saved job list. 2. The saved job posting is associated with the applicant for future access. 3. No application is created or submitted.
Main Flow	1. The applicant requests to view an available job posting. 2. The system provides the chosen job posting and its details. 3. The applicant requests to save the job posting. 4. The system verifies that the job posting has not already been saved by the applicant. 5. The system adds the job posting to the applicant's saved job list. 6. The system confirms that the job posting has been saved successfully.

Extensions	4a. The job posting has already been saved by the applicant. 4a1. The system reports that the job posting is already in the applicant's saved job list. 4a2. No duplicate saved job entry is created. 4b. The job posting is no longer available. 4b1. The system reports that the job posting is no longer available for saving. 4b2. The job posting is not added to the applicant's saved job list.
Special Requirements	● A job posting may be saved only once by the same applicant. ● Saved job postings must be accessible only to the applicant who saved them. ● Saving a job posting does not create or submit a job application. ● If a saved posting closes later, it stays in the saved list. It is marked as closed.

4.2.1.7 Use Case Name: Track Application Stage

Use Case Name	**Track Application Stage**
Author	**Spencer Hammack**
Actor	**Applicant**
Preconditions	1. The applicant is identified by the system.
Postconditions	1. The applicant has viewed the current stage of the chosen application. 2. No application is created or changed.
Main Flow	1. The applicant requests to view their submitted applications. 2. The system retrieves the applicant's applications. 3. The system provides each application and its current stage. 4. The applicant chooses an application. 5. The system provides the chosen application details and current stage. 6. The system provides the application's stage history.
Extensions	2a. The applicant has no submitted applications. 2a1. The system reports that the applicant has no submitted applications. 4a. The applicant does not choose an application. 4a1. The system continues to provide the applicant's applications. 6a. No stage history is available for the application. 6a1. The system provides the application's current stage without additional history.
Special Requirements	● Only the applicant's own applications are provided.

4.2.1.8 Use Case Name: Withdraw an Application

Use Case Name	**Withdraw an Application**
Author	**Spencer Hammack**
Actor	**Applicant**
Preconditions	1. The applicant is identified by the system. 2. The applicant has submitted an application.
Postconditions	1. The application is in the WITHDRAWN stage. 2. The application is out of the active recruiting process. 3. No other application is changed.
Main Flow	1. The applicant starts to withdraw an application. 2. The system provides the application and its details. 3. The applicant requests to withdraw the application. 4. The system verifies that the application is in the SUBMITTED, UNDER_REVIEW, or INTERVIEW stage. 5. The system asks the applicant to confirm the withdrawal. 6. The applicant confirms the withdrawal. 7. The system records the application as WITHDRAWN. 8. The system reports that the application has been successfully withdrawn.
Extensions	4a. The application is not in the SUBMITTED, UNDER_REVIEW, or INTERVIEW stage. 4a1. The system reports that the application is past the stages that allow withdrawal and keeps it unchanged. 5a. The applicant ends the use case before confirming the withdrawal. 5a1. The application remains unchanged.
Special Requirements	● The system records the withdrawn application. ● A withdrawn application does not continue through the active recruiting process.

4.2.2 Actor Name: Recruiter

4.2.2.1 Use Case Name: Create and Publish a Job Posting

Use Case Name	**Create and Publish a Job Posting**
Author	**Luyu Niu**
Actor	**Recruiter**

Preconditions	1. The recruiter is identified by the system. 2. The recruiter belongs to a company that exists in the system.
Postconditions	1. A job posting exists with status PUBLISHED and belongs to the recruiter's company. 2. The posting is open for applications until its closing date. 3. No other posting is created or changed.
Main Flow	1. The recruiter starts a new job posting. 2. The system asks for the posting details. 3. The recruiter provides the title, description, location, employment type and closing date. 4. The system validates the details. 5. The recruiter requests publication of the posting. 6. The system records the posting as published, with the recruiter and the publication time. 7. The system reports the published posting and its details.
Extensions	4a. A required detail is missing. 4a1. The system reports which detail is missing and asks for it. 4b. The closing date is not later than the current date. 4b1. The system reports the problem and asks for a later closing date. 5a. The recruiter saves the posting as a draft instead of publishing it. 5a1. The system saves the draft with the recruiter and the creation time. 5a2. The recruiter later resumes the saved draft and continues at step 5. 5a3. The closing date has passed when the draft is resumed. The system asks for a new closing date and repeats steps 4 to 7. 5b. The recruiter ends the use case before publication. 5b1. No posting is published; a draft saved in 5a stays a draft.
Special Requirements	● The system records the recruiter and the time for the creation and for the publication. ● A posting that is still a draft is not returned by applicant searches.

4.2.2.2 Use Case Name: Manage Job Posting

Use Case Name	**Manage Job Posting**
Author	**Yuting Mei**
Actor	**Recruiter**
Preconditions	1. The recruiter is identified by the system. 2. The recruiter belongs to a company that exists in the system. 3. The recruiter's company has at least one existing job posting.

Postconditions	1. The chosen job posting is updated with the confirmed changes. 2. The updated job posting and its current status are recorded by the system. 3. No application is created, changed or deleted.
Main Flow	1. The recruiter requests to manage an existing job posting. 2. The system retrieves the job postings belonging to the recruiter's company. 3. The system provides the job postings and their current statuses. 4. The recruiter chooses a job posting. 5. The system provides the chosen job posting and its current details. 6. The recruiter modifies the job posting information. 7. The system validates the modified information. 8. The recruiter confirms the changes. 9. The system saves the updated job posting. 10. The system confirms that the job posting has been updated.
Extensions	4a. The recruiter does not choose a job posting. 4a1. The system continues to provide the recruiter's job postings. 6a. The recruiter requests to close the job posting. 6a1. The system asks the recruiter to confirm the closing action. 6a2. The recruiter confirms the action. 6a3. The system changes the job posting status to *CLOSED*. 6a5. Existing applications associated with the job posting remain unchanged. 6a6. The system confirms that the job posting has been closed. 7a. A required job posting detail is missing or invalid. 7a1. The system reports the invalid or missing information and asks the recruiter to correct it. 7b. The recruiter provides a closing date that has already passed. 7b1. The system reports the problem and asks for a valid closing date. 8a. The recruiter does not confirm the changes. 8a1. The existing job posting remains unchanged.
Special Requirements	● Only authorized recruiters belonging to the company may modify its job postings. ● Changes to a job posting must not modify or delete existing applications associated with the posting. ● A *CLOSED* job posting must not accept new applications or appear in searches for available jobs.

4.2.2.3 Use Case Name: Review Submitted Applications

Use Case Name	**Review Submitted Applications**

Author	**Peter Chen**
Actor	**Recruiter**
Preconditions	1. The recruiter is identified by the system. 2. The recruiter belongs to a company that exists in the system.
Postconditions	1. The recruiter has reviewed the chosen application. 2. No other changes occur to the application.
Main Flow	1. The recruiter requests to review submitted applications. 2. The system retrieves the applications submitted to the recruiter's job postings. 3. The system provides the submitted applications. 4. The recruiter chooses an application. 5. The system retrieves the chosen application's details. 6. The system provides the application details. 7. The recruiter reviews the provided information.
Extensions	2a. No application has been submitted to the recruiter's job postings. 2a1. The system reports that no application has been submitted. 4a. The recruiter does not choose an application. 4a1. The system continues to provide the submitted applications. 5a. The chosen application is no longer available. 5a1. The system reports that the chosen application is unavailable. 5a2. The system continues to provide the available submitted applications.
Special Requirements	● Only applications submitted to job postings associated with the recruiter's company are provided.

4.2.2.4 Use Case Name: Update Application Stage

Use Case Name	**Update Application Stage**
Author	**Peter Chen**
Actor	**Recruiter**
Preconditions	1. The recruiter is identified by the system. 2. The recruiter belongs to a company that exists in the system.
Postconditions	1. The chosen application's stage is updated. 2. The updated application stage is recorded by the system.

Main Flow	1. The recruiter requests to update an application stage. 2. The system retrieves the applications submitted to the recruiter's job postings. 3. The system provides the applications and their current stages. 4. The recruiter chooses an application. 5. The system provides the chosen application's current stage and available next stages. 6. The recruiter chooses a new stage. 7. The recruiter confirms the stage change. 8. The system updates and records the application stage.
Extensions	2a. No application has been submitted to the recruiter's job postings. 2a1. The system reports that no application has been submitted. 4a. The recruiter does not choose an application. 4a1. The system continues to provide the submitted applications. 6a. The recruiter does not choose a new stage. 6a1. The application stage remains unchanged. 7a. The recruiter does not confirm the stage change. 7a1. The application stage remains unchanged.
Special Requirements	● Only authorized recruiters can update the stage of applications submitted to their company's job postings. ● The application stage remains unchanged if the requested change is invalid or not confirmed. ● Final hiring decisions are represented by final application stages, such as ACCEPTED, DECLINED or REJECTED.

4.2.2.5 Use Case Name: Extend Job Offer

Use Case Name	**Extend Job Offer**
Author	**Spencer Hammack**
Actor	**Recruiter**
Preconditions	1. The recruiter is identified by the system. 2. The recruiter belongs to a company that exists in the system.
Postconditions	1. The chosen application is in the OFFER stage. 2. A new StageRecord records the OFFER stage, the date of the change, and the recruiter responsible. 3. Existing stage history is preserved. 4. No other application is changed.
Main Flow	1. The recruiter requests to view applications submitted to their company's job postings. 2. The system retrieves the applications. 3. The system provides the applications and their current stages. 4. The recruiter chooses an application. 5. The system provides the selected application's details and current stage. 6. The recruiter requests to record that a job offer has been extended.

	7. The system verifies that the application is in the UNDER_REVIEW or INTERVIEW stage. 8. The system asks the recruiter to confirm the stage change. 9. The recruiter confirms the stage change. 10. The system records the application stage as OFFER and appends a StageRecord containing the stage, date, and responsible recruiter. 11. The system confirms that the offer stage has been recorded.
Extensions	2a. No applications exist for the company's job postings. 2a1. The system reports that no applications are available. 4a. The recruiter does not choose an application. 4a1. The system continues to provide the available applications. 7a. The application is not in the UNDER_REVIEW or INTERVIEW stage. 7a1. The system reports that the application is not in the UNDER_REVIEW or INTERVIEW stage. 7a2. The application and its stage history remain unchanged. 9a. The recruiter cancels or leaves before confirming the stage change. 9a1. The application and its stage history remain unchanged.
Special Requirements	● Only authorized recruiters may update applications submitted to their company's job postings. ● Every successful stage change must preserve the application's previous stage history and record the date and responsible user. ● Offer communication and negotiation take place outside the system. This use case records only the resulting OFFER stage. ● An invalid or unconfirmed stage change must leave the application unchanged.

4.2.3 Actor Name: Administrator

4.2.3.1 Use Case Name: Manage Accounts and Roles

Use Case Name	**Manage Accounts and Roles**
Author	**Luyu Niu**
Actor	**Administrator**
Preconditions	1. The administrator is identified by the system.
Postconditions	1. The account holds the role and the status the administrator assigned. 2. No other account is changed. 3. The change is recorded with the administrator and the time.

Main Flow	1. The administrator starts an account change. 2. The system asks which account to change. 3. The administrator identifies the account. 4. The system reports the current role and the current status of the account. 5. The administrator assigns a new role, activates the account, or deactivates it. 6. The system validates the change. 7. The system applies the change and records it with the administrator and the time. 8. The system reports the new role and the new status of the account.
Extensions	3a. No account matches the identification. 3a1. The system reports that no account matched and asks for another account. 6a. The change would leave the system without an active administrator. 6a1. The system rejects the change and keeps the current role. 6b. The account is already in the requested status. 6b1. The system reports the current status and asks for a different change. 6c. The account still owns applications, resumes, job postings or recorded account changes. 6c1. The system rejects the role change, names the records, and suggests deactivating the account instead. 6d. The new role is Recruiter and no company is given. 6d1. The system asks for the company the recruiter works for. 7a. The account holder has an active session. 7a1. The system ends that session.
Special Requirements	● Every change of role or status is recorded with the administrator and the time. ● A deactivated account is refused at sign in.

5. Domain Model (Analysis)

We listed the nouns in our requirements and use cases. We kept a noun as a concept if it exists in the business without the software and matters to recruiting, not only to one screen. Other nouns became attributes, such as the closing date. Table 1 shows the concepts we kept and what kind of thing each one is.

Figure 2. Domain model of the recruiting system

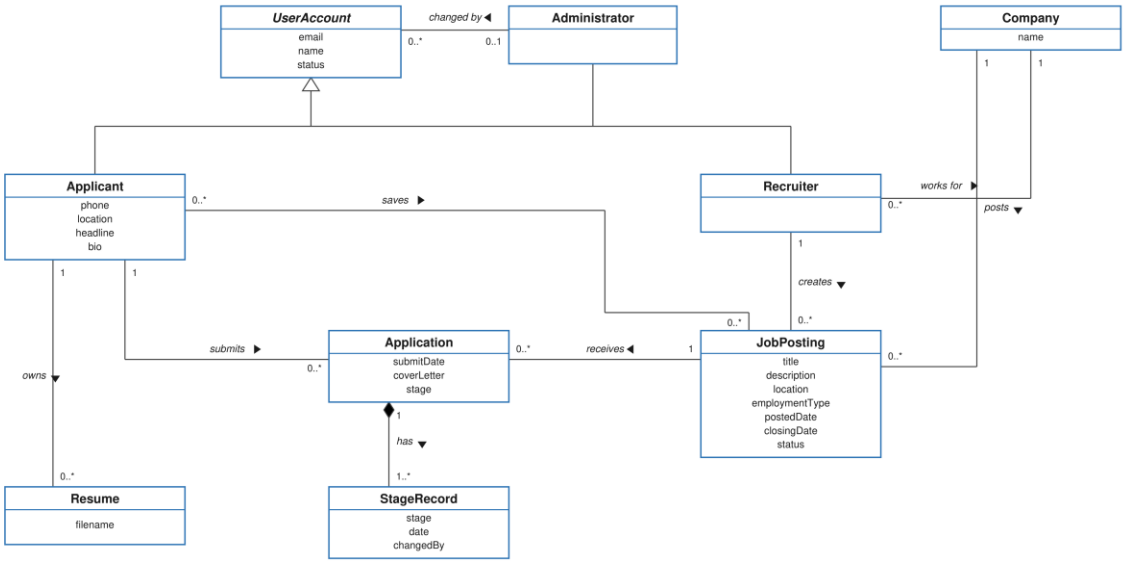


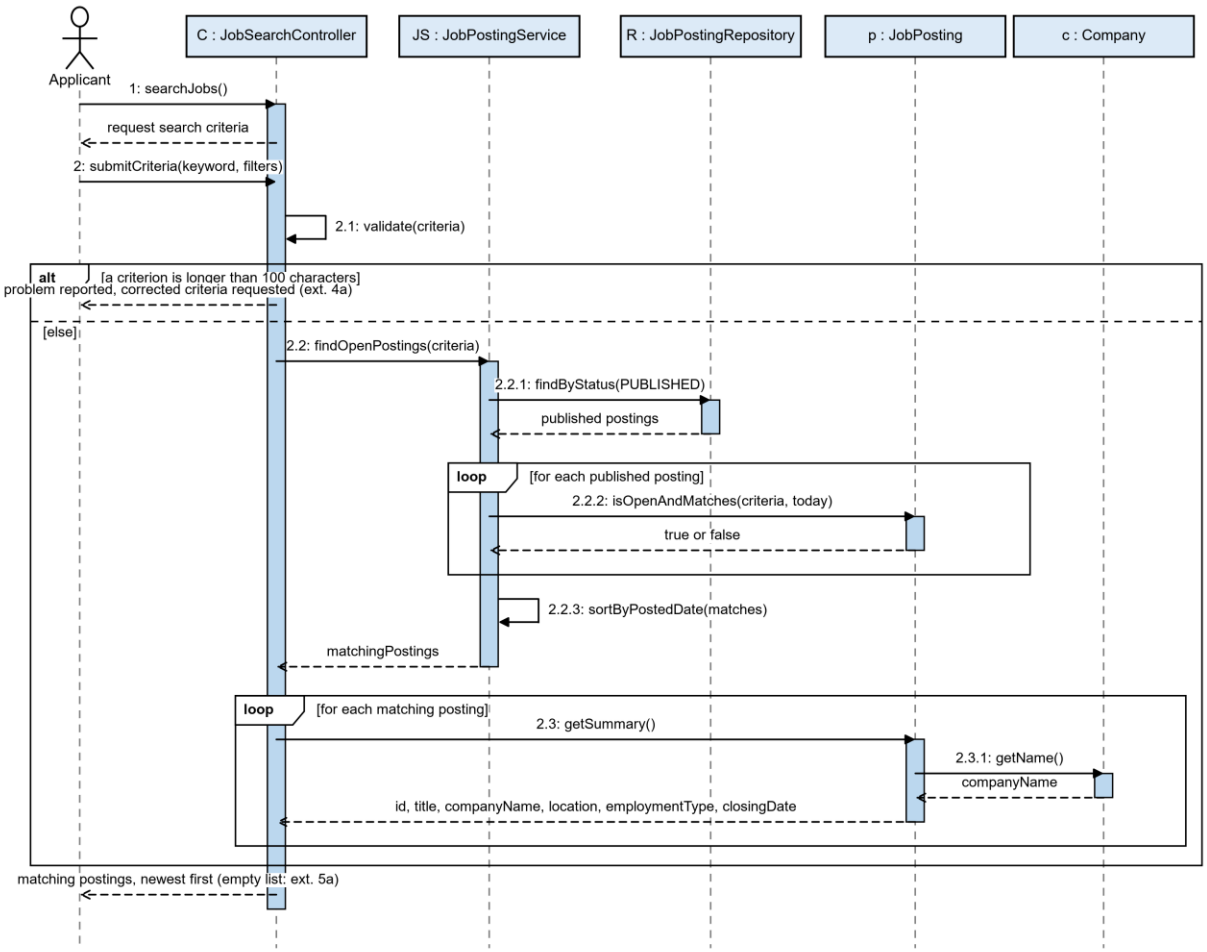
Table 1. Domain concepts and the kind of thing each one is

Kind	Concept
People and roles	Applicant, Recruiter, Administrator
Business entity	Company
Record	JobPosting, UserAccount
Physical object	Resume
Transaction	Application
Event	StageRecord

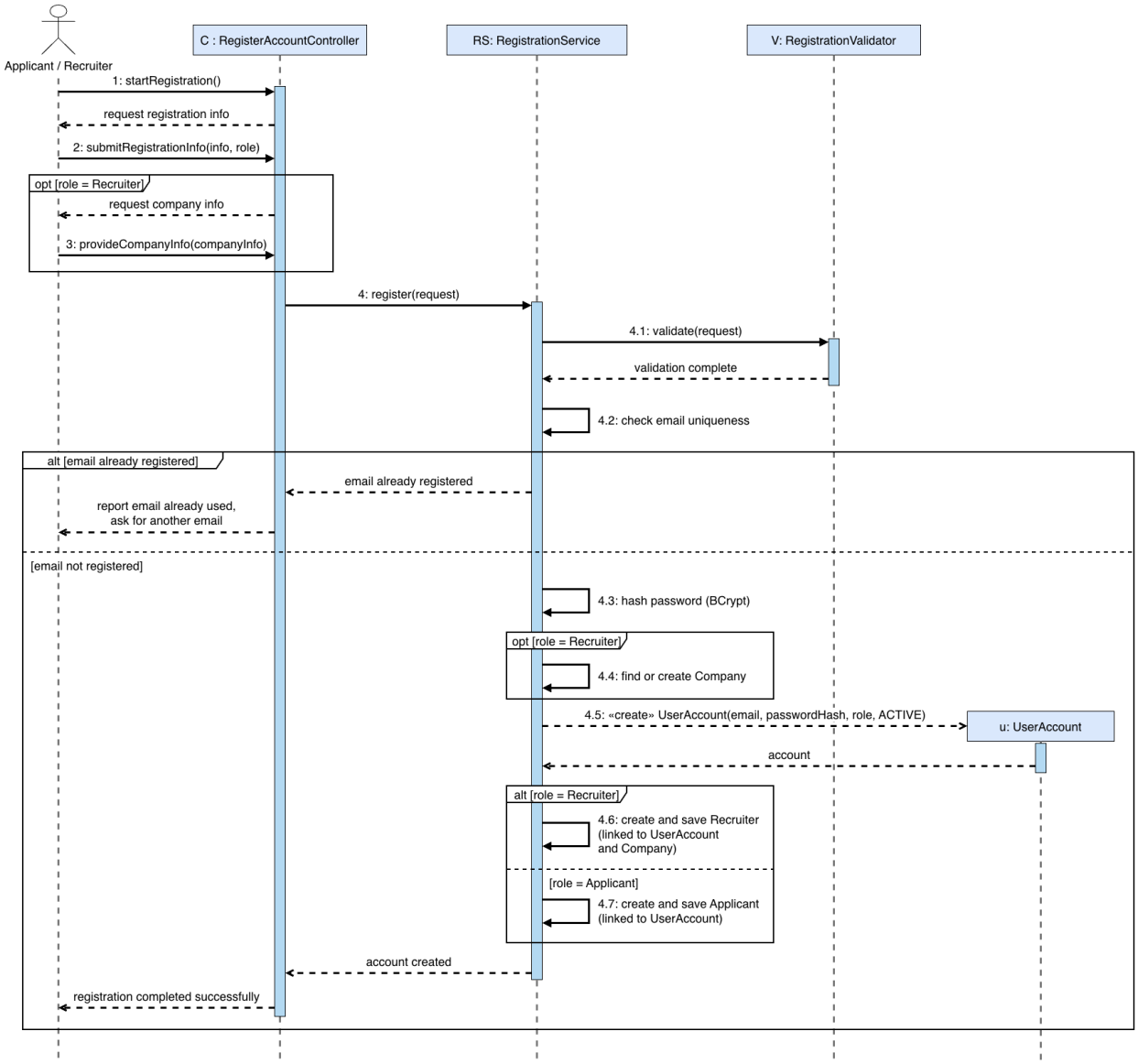
The Iteration 1 review changed the model in three places. UserAccount is an abstract class with the attributes every user shares. Applicant, Recruiter and Administrator inherit from it, so one account has exactly one role. A Resume belongs to an Applicant. An Application reaches the resume through the applicant and has no direct link to it. StageRecord is the history of one application. Each record names a stage, the date it was entered and who made the change. The records exist only as part of their application. The filled diamond (composition) shows this.

6. Sequence Diagrams (Design)

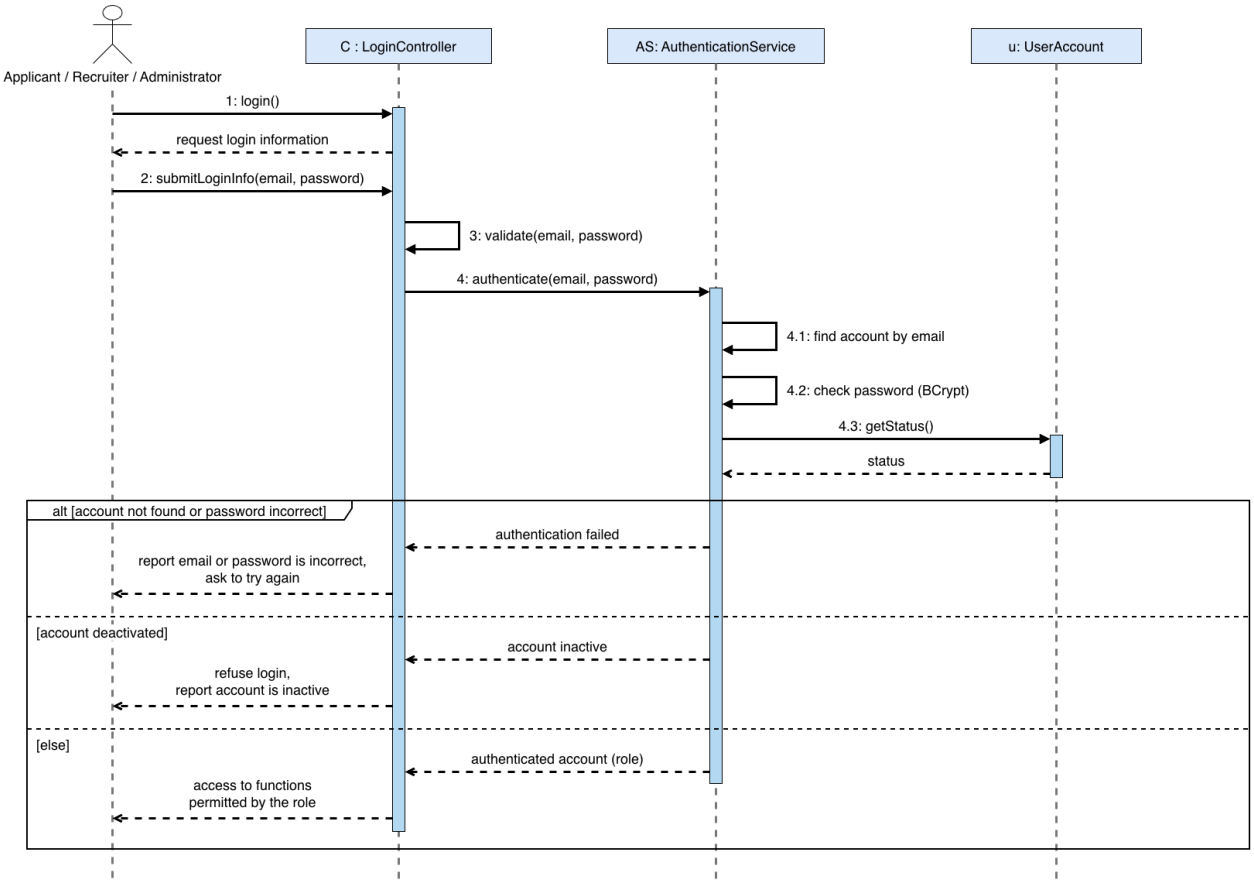
6.1 Use Case Name: Search Available Jobs (Luyu Niu)



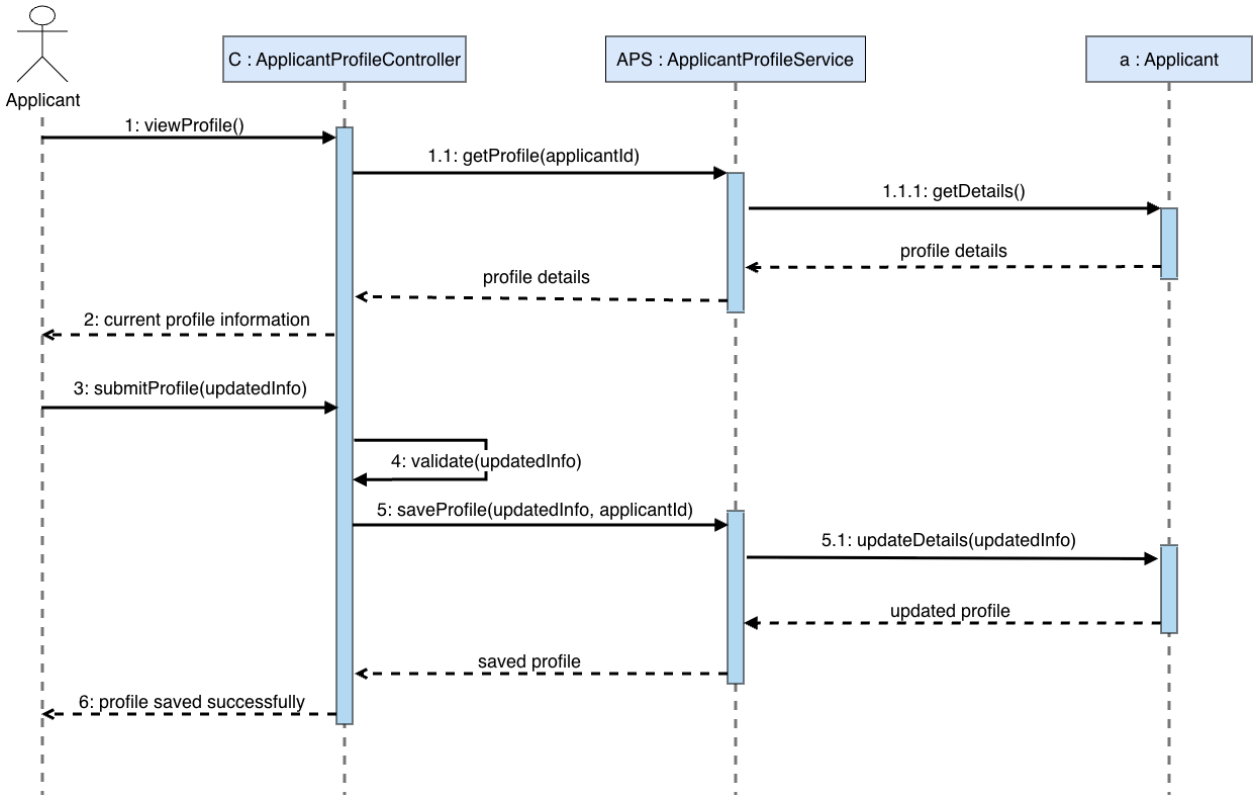
6.2 Use Case Name: Register Account (Xinyi Chen)



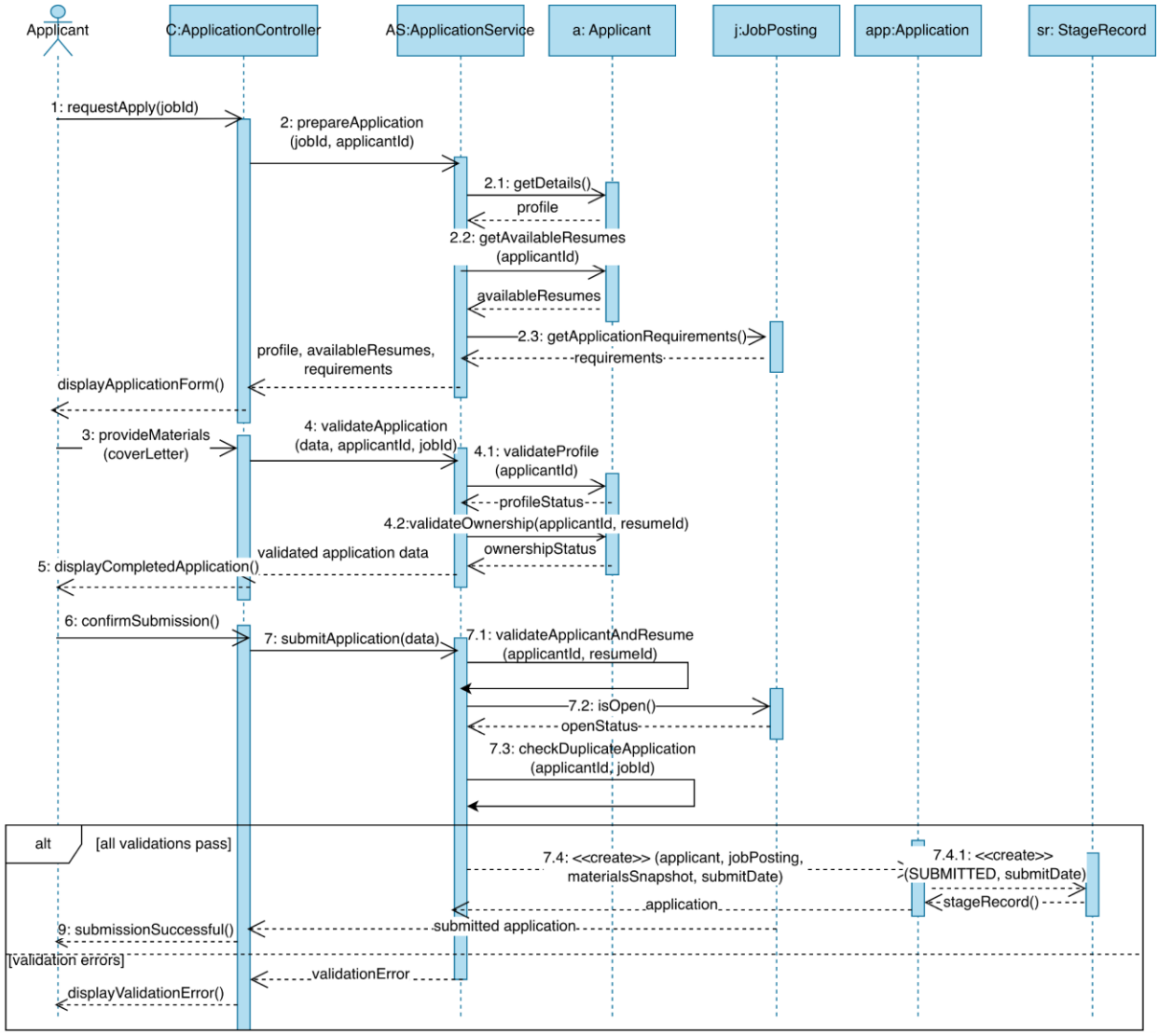
6.3 Use Case Name: User Login (Xinyi Chen)



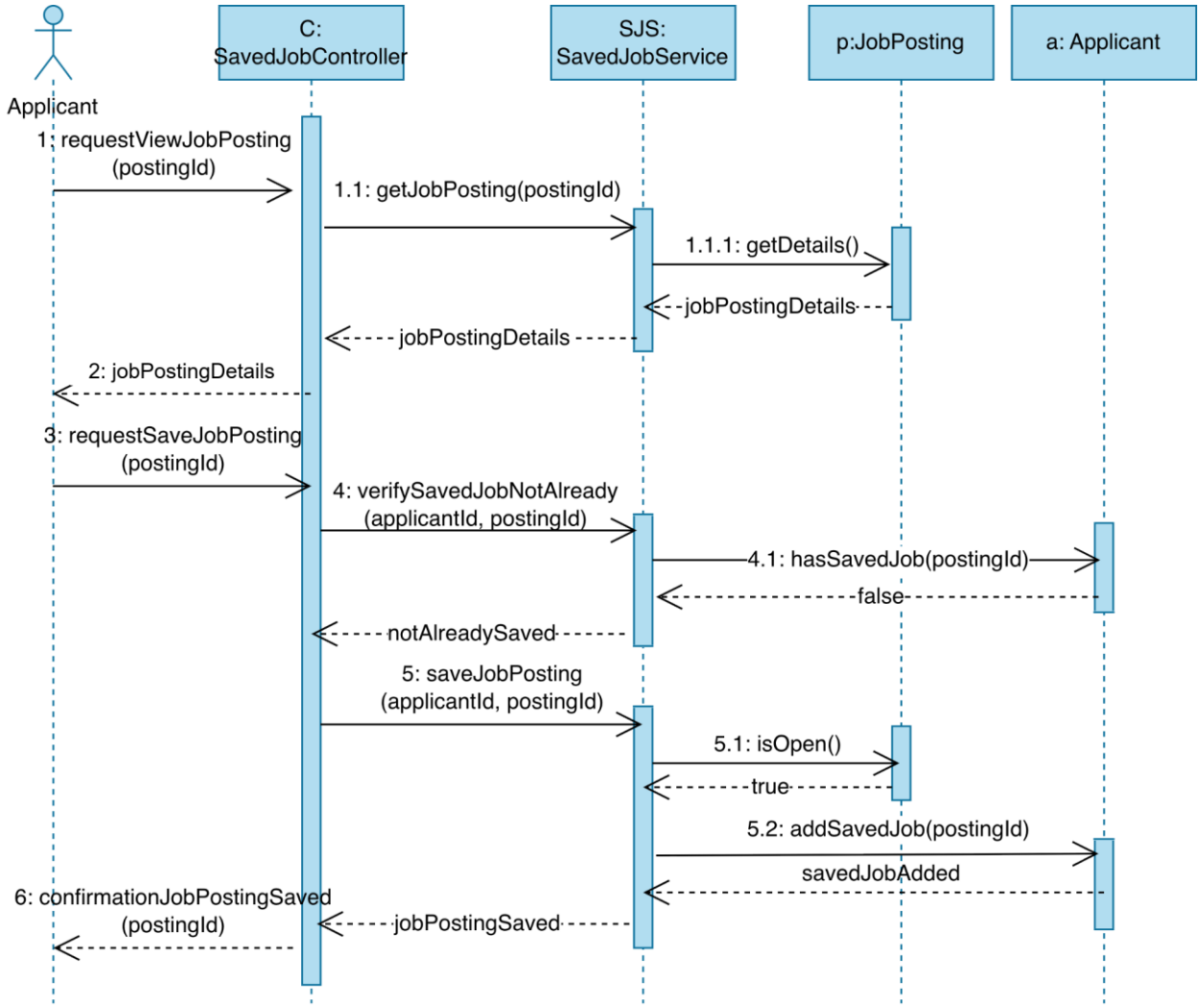
6.4 Use Case Name: Update Applicant Profile (Xinyi Chen)



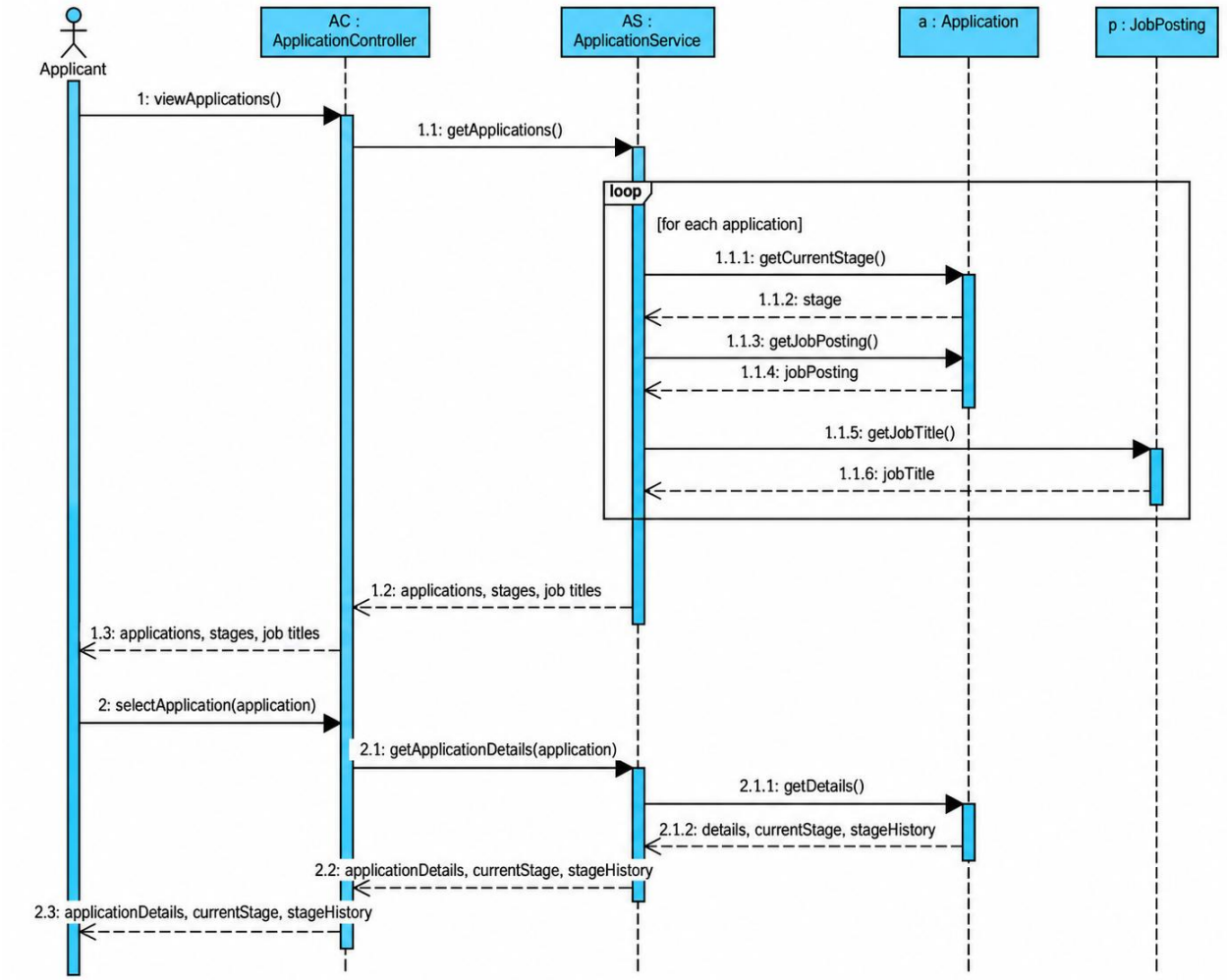
6.5 Use Case Name: Submit Job Application (Yuting Mei)



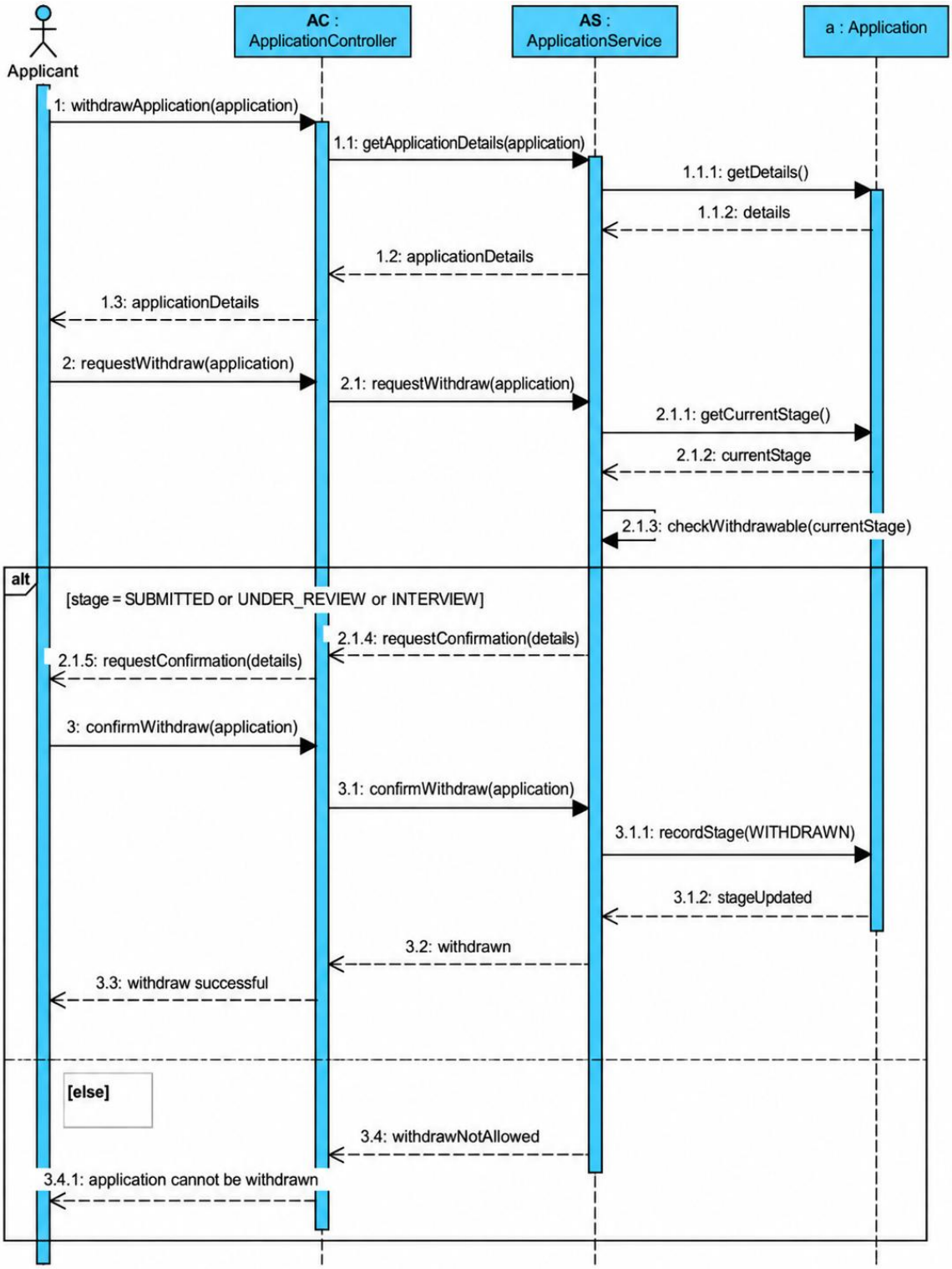
6.6 Use Case Name: Save Job Posting (Yuting Mei)



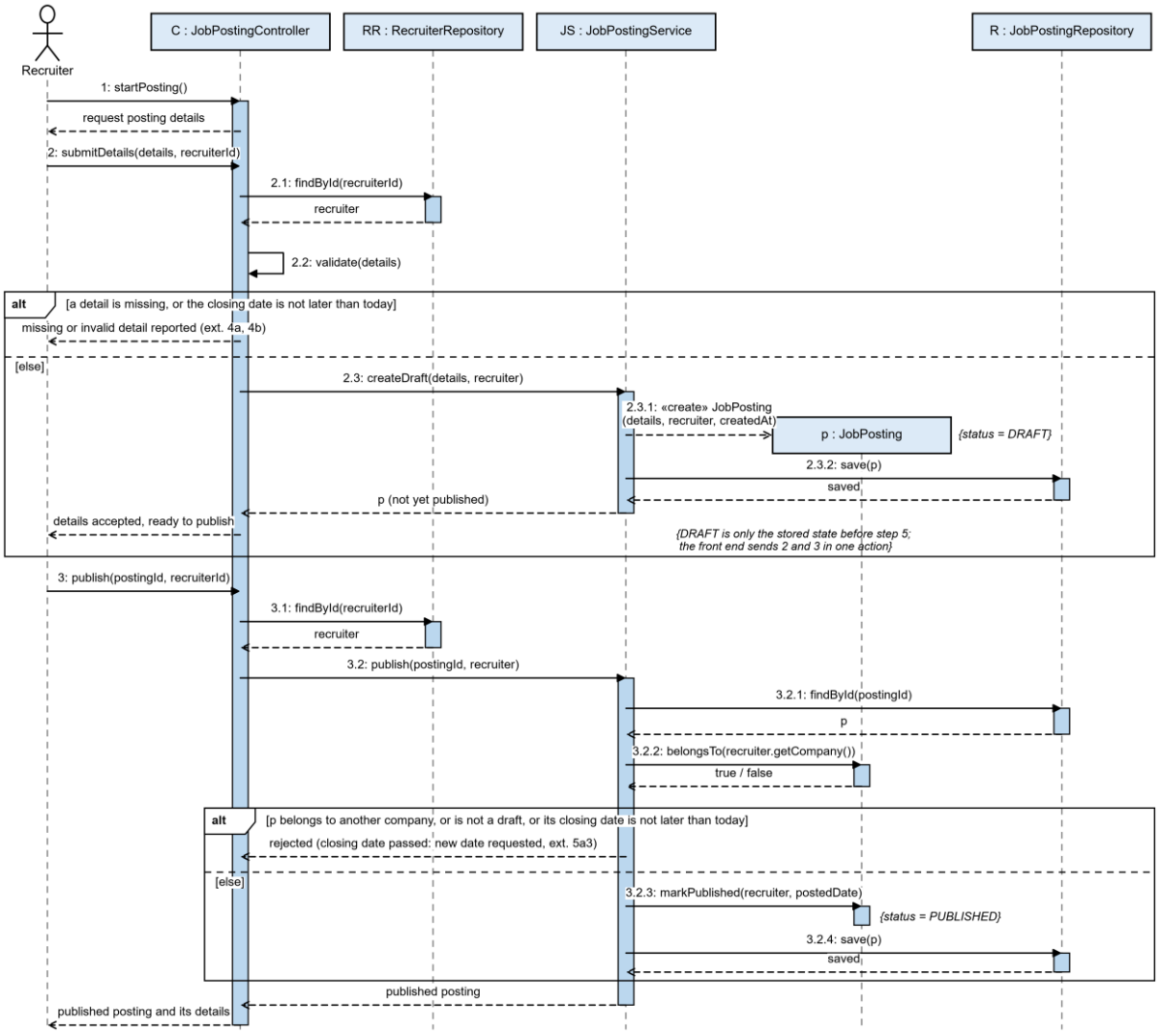
6.7 Use Case Name: Track Application Stage (Spencer Hammack)



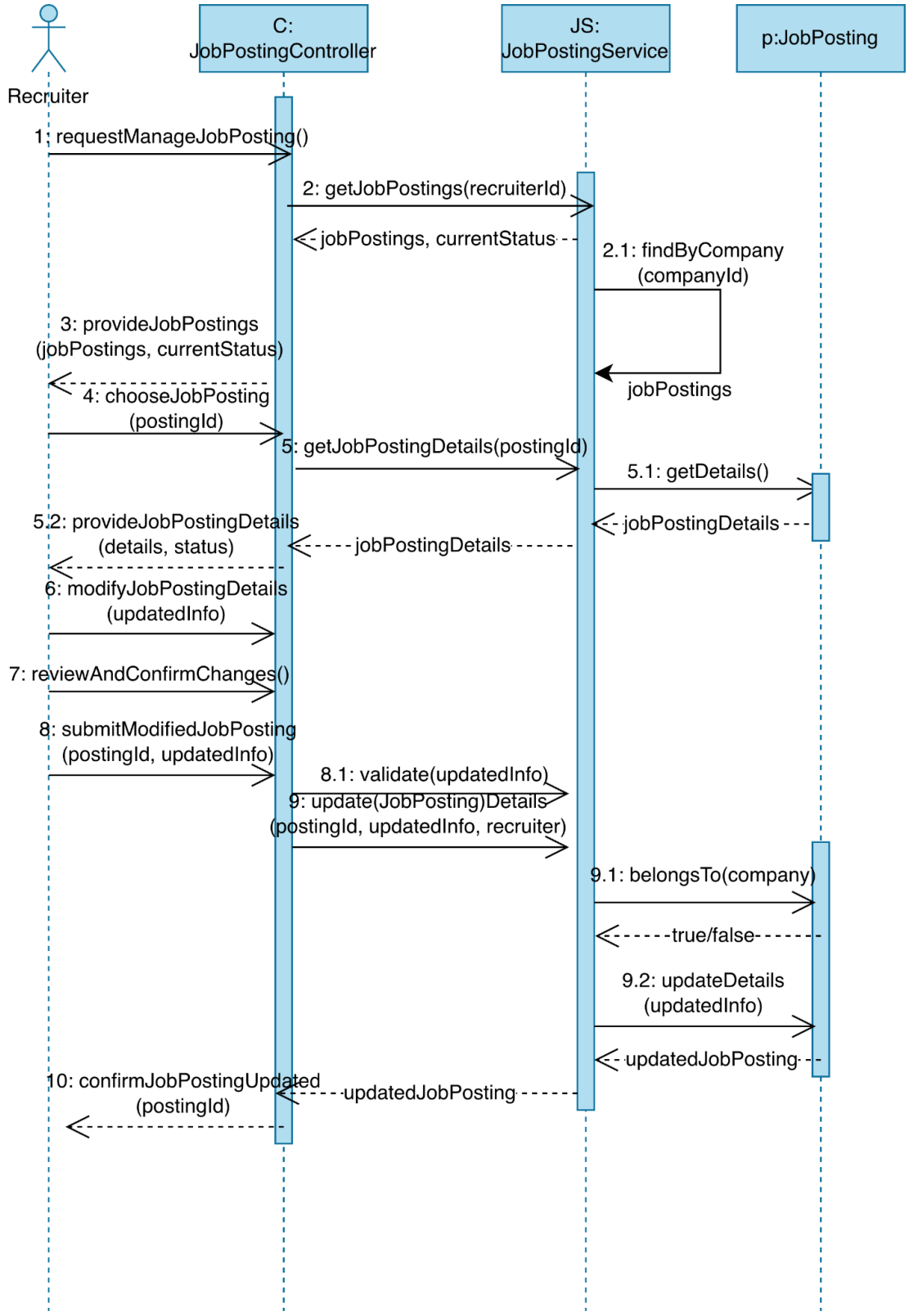
6.8 Use Case Name: Withdraw an Application (Spencer Hammack)



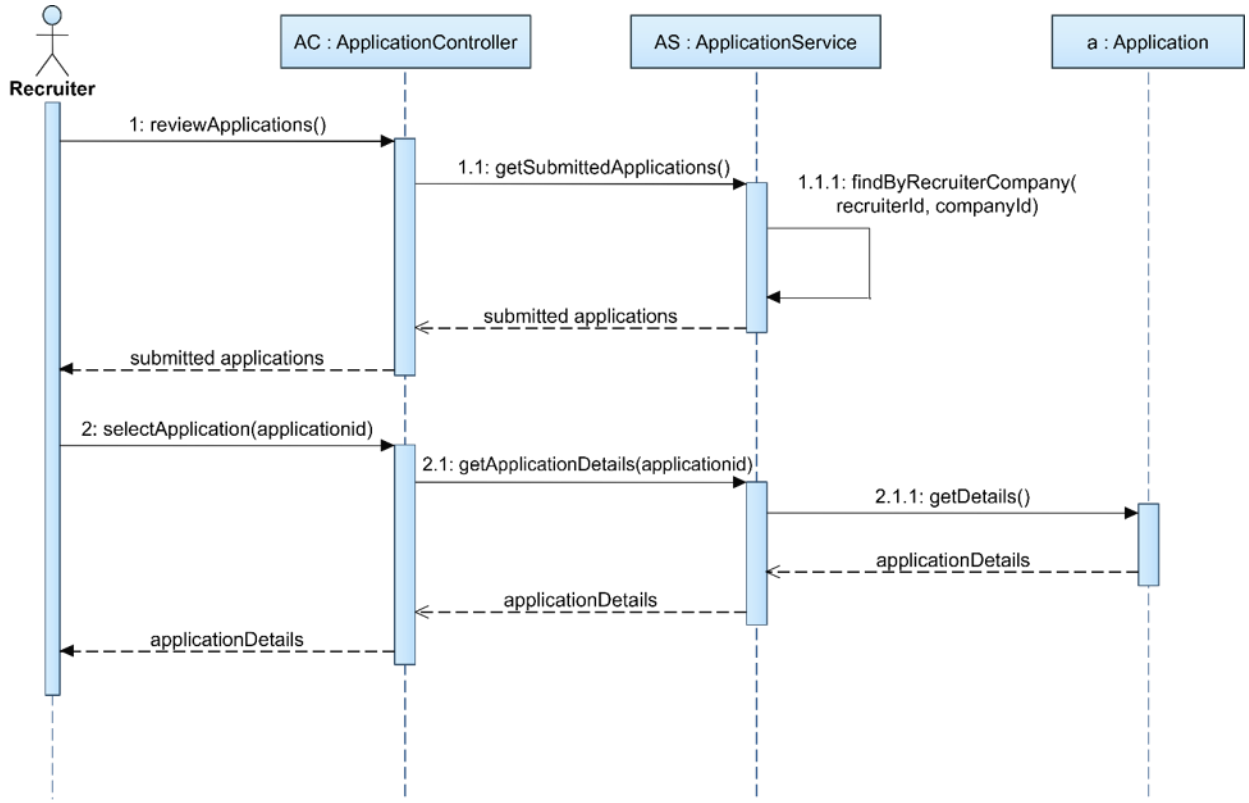
6.9 Use Case Name: Create and Publish a Job Posting (Luyu Niu)



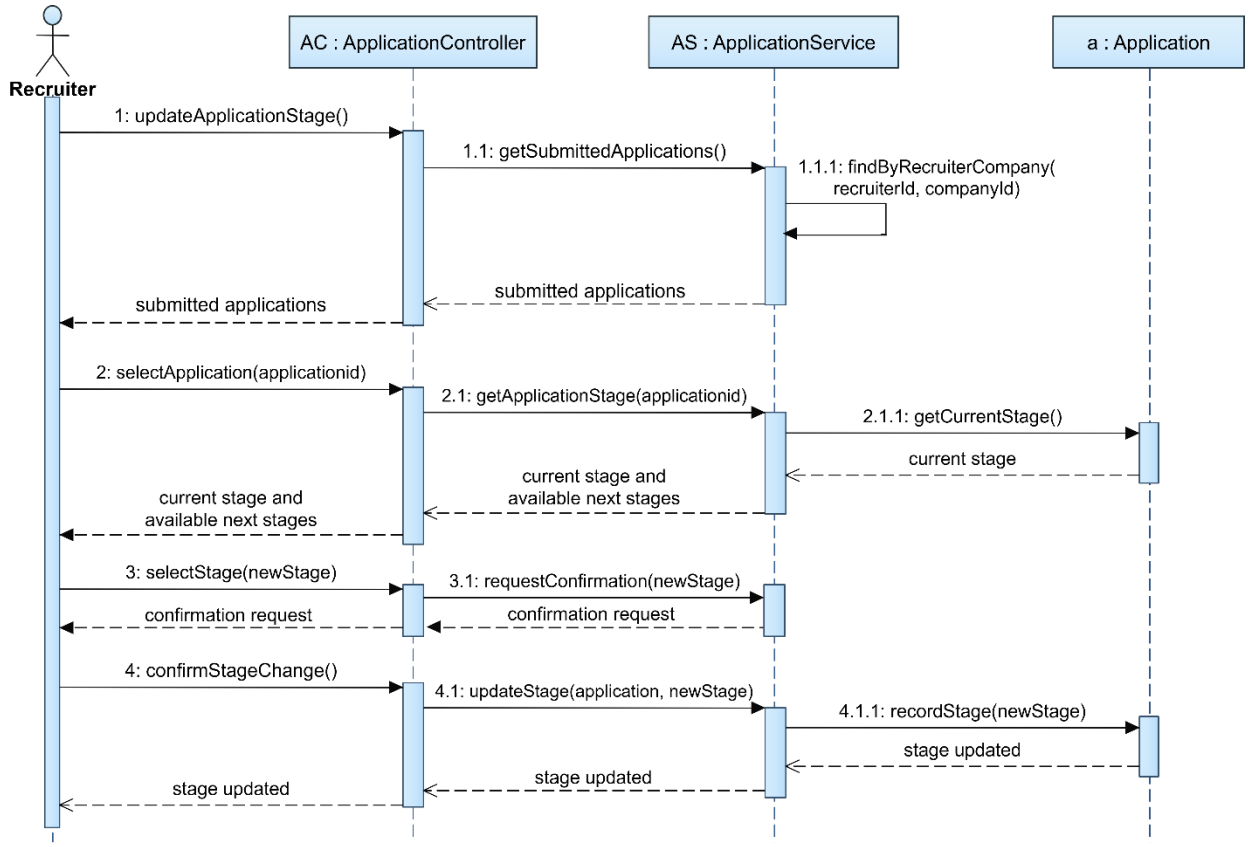
6.10 Use Case Name: Manage Job Posting (Yuting Mei)



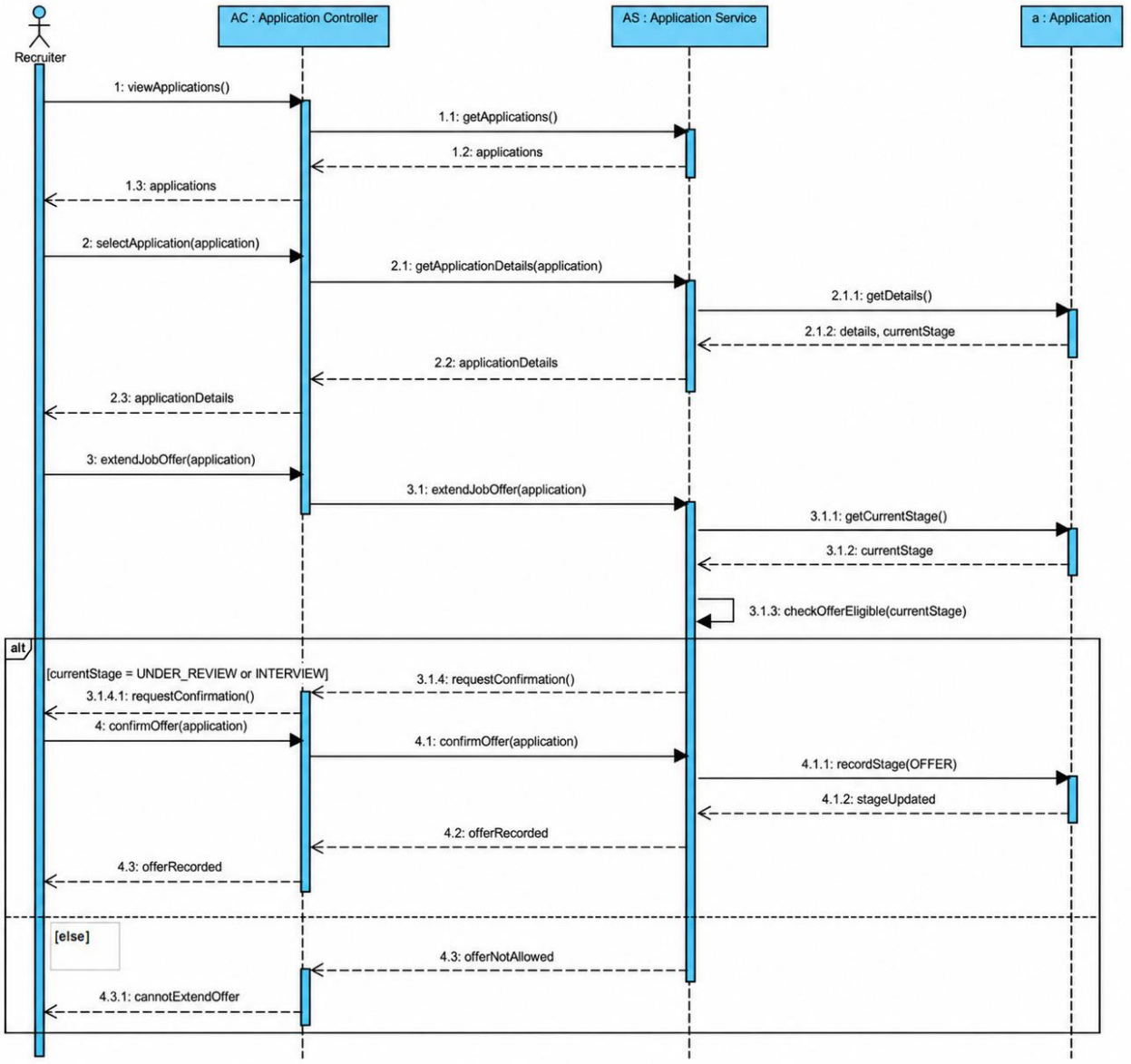
6.11 Use Case Name: Review Submitted Applications (Peter Chen)



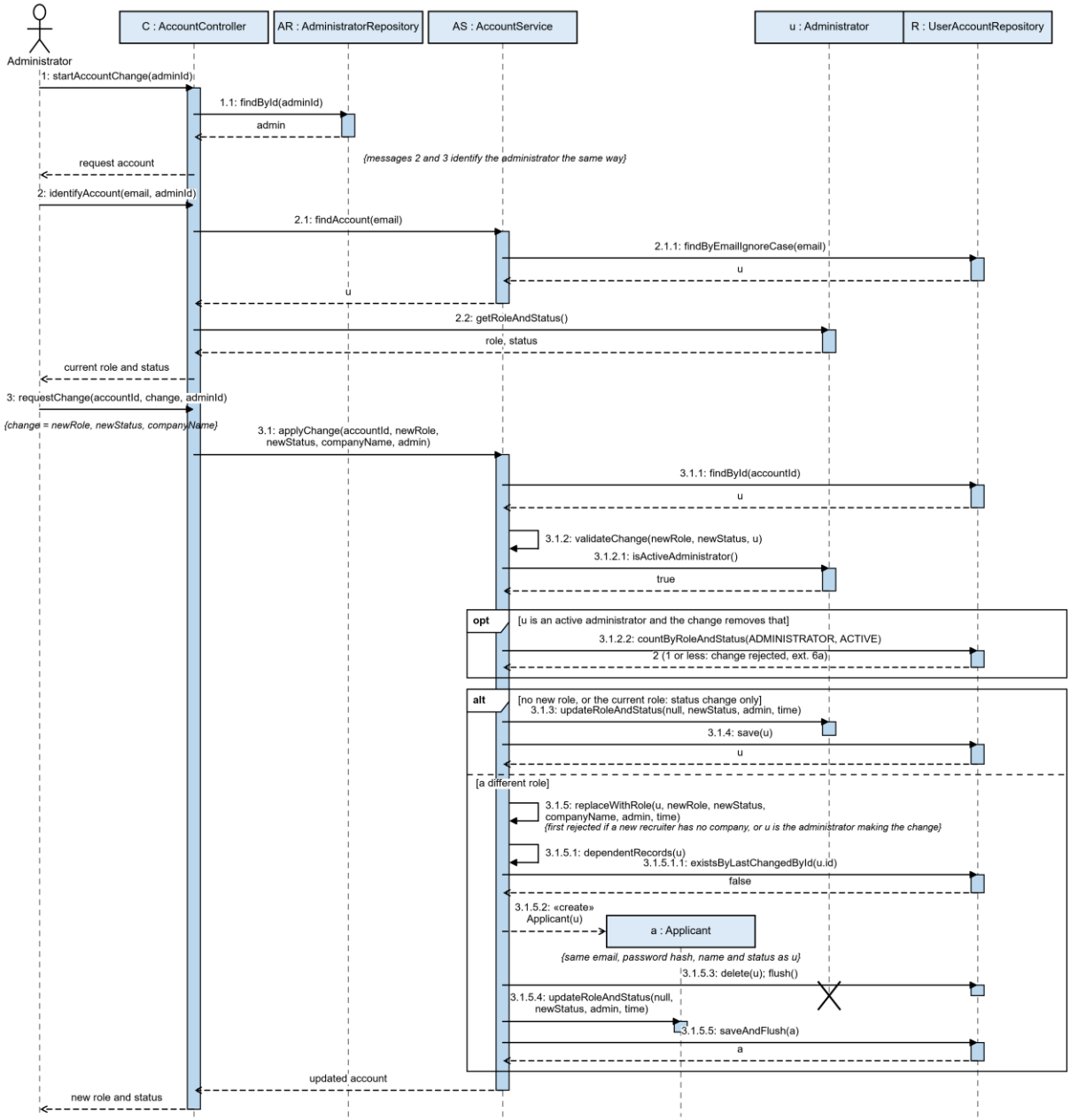
6.12 Use Case Name: Update Application Stage (Peter Chen)



6.13 Use Case Name: Extend Job Offer (Spencer Hammack)



6.14 Use Case Name: Manage Accounts and Roles (Luyu Niu)



7. Design Class Diagram (DCD)

Figure 3 is the design class diagram (DCD) of the whole back end on the main branch. It builds on the domain model of Section 5. The middle band holds the entity classes. They are placed and linked as in Figure 2, now with their attributes and the operations the sequence diagrams call. Above them are the controllers (web layer) and the services. Below them are the repositories, the classes that read and write the database. Dashed arrows are dependencies, labeled with the operations called. Solid lines with an open arrowhead are navigable associations with their multiplicities. A hollow triangle is a generalization. As in the domain model, UserAccount is an abstract superclass. Applicant, Recruiter and Administrator inherit from it (see the inheritance decision in Section 8.2).

The request flow is the one of Section 8.1: controller, one service, entities, repositories. Four controllers also use repositories directly. AccountController and JobPostingController load the acting administrator or recruiter. ResumeController lists and uploads resumes. DemoIntegrationController only prepares data for the React pages and is left out of the figure. LoginController and AuthenticationService implement User Login. ApplicantProfileController and ApplicantProfileService implement Update Applicant Profile. StageRecord is the embedded stage history of Application.

Two things of Figure 2 are not entity links yet. The saves association (Applicant to JobPosting) is a list of posting ids that DemoIntegrationController keeps in a file (Table 4, row 4.2.1.6). The resume of an application is a copy stored in the Application (submittedResume), reached through the applicant at submission. Getters, the HttpServletRequest parameters of the login and profile endpoints and the CurrentUser helper are left out. Section 8.3 groups the same classes into subsystems.

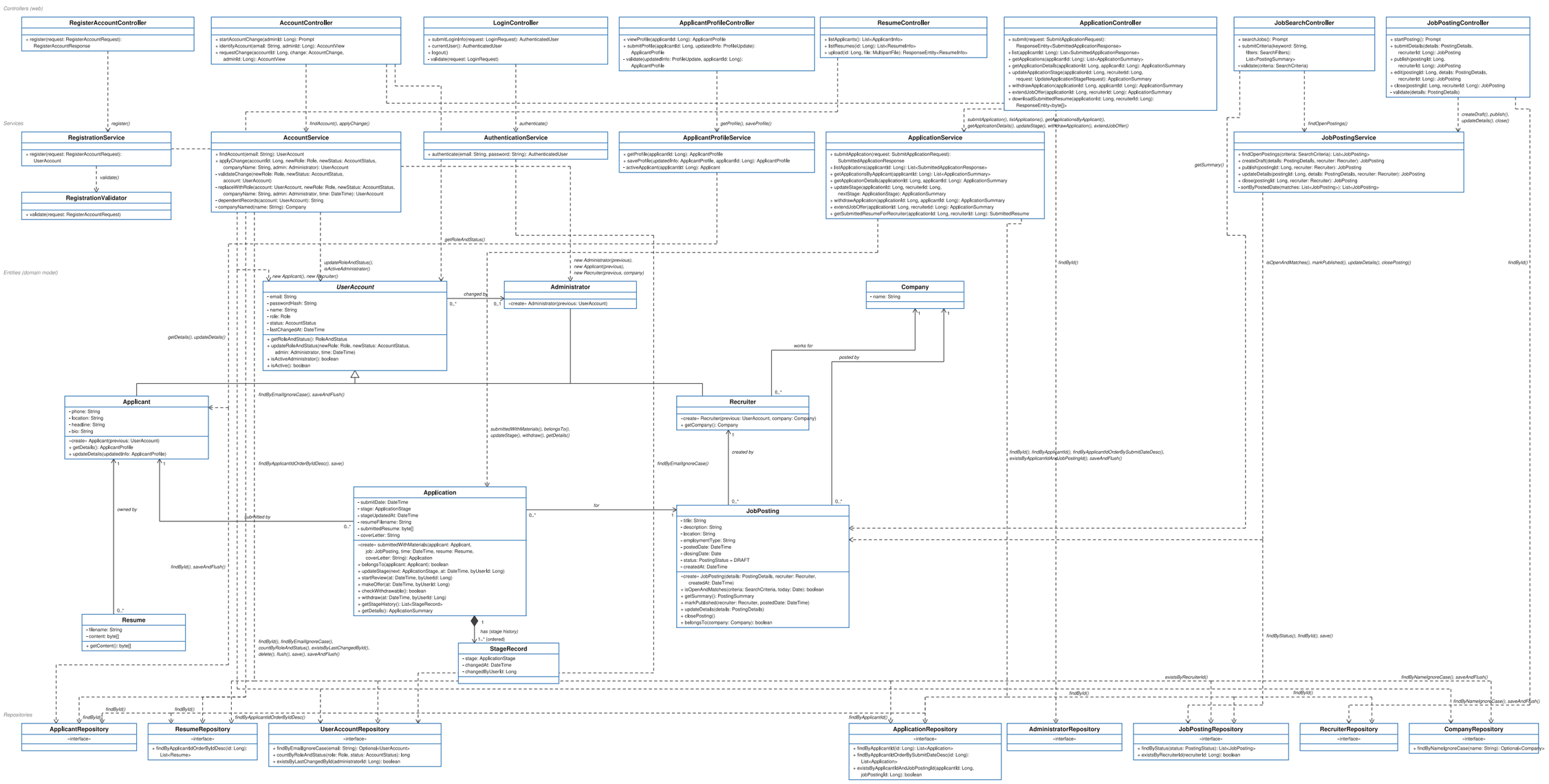


Figure 3. Design class diagram of the back end: the domain model as frame, with controllers, services and repositories

8. Implementation Overview

The system is a Java 21 back end built with Spring Boot 4 and Spring Data JPA. It runs on an H2 database (a small database that runs inside the application) during development. Maven builds it and JUnit 5 runs the tests. The user interface is a React application (frontend folder, built with Vite). It calls the REST endpoints, the web addresses the pages use. Two plain HTML pages served by the back end remain for Submit Job Application and My Applications. The code lives in the group repository at https://github.com/baylor-csi5324-team3/recruiting-system.

Every member works on a feature branch. Changes reach the main branch only through a reviewed pull request.

8.1 Architecture

The back end has four layers plus configuration classes, under the package edu.baylor.csi5324.rams. A request enters a controller in the web layer. The controller calls one service. The service applies the business rules with the domain objects and reads or writes them through a repository. Each layer only calls the layer below it.

Table 2. Layers of the back end

Layer	Package	Main classes	Responsibility
Web (UI and API)	web	JobSearchController, JobPostingController, AccountController, ApplicationController, RegisterAccountController, ResumeController, LoginController, ApplicantProfileController, DemoIntegrationController, IdentityCheckInterceptor, ApiExceptionHandler and the other exception handlers	Receives HTTP requests, validates the input, calls one service, and returns JSON or an error. The two static pages submit-application.html and applications.html call these controllers.
Service	service	JobPostingService, AccountService, ApplicationService, RegistrationService, RegistrationValidator, AuthenticationService, ApplicantProfileService	One class per group of related use cases. Holds the business rules and the transaction boundary. Throws ValidationException (400) and NotFoundException (404).
Domain	domain	UserAccount, Administrator, Recruiter, Company, JobPosting, Applicant, Application, Resume; embeddable StageRecord; enumerations Role, AccountStatus, PostingStatus, ApplicationStage; records such as PostingDetails and ApplicationSummary	JPA entities that keep the names and associations of the domain model. Rules that belong to one object live in that object, for example JobPosting.isOpenAndMatches and Application.updateStage.
Repository	repository	One Spring Data JPA interface per entity	Persistence. We write no SQL. Spring builds each query from the method name.

Layer	Package	Main classes	Responsibility
Configuration	config	ClockConfig, DevDataSeeder, JobCsvParser, JobCsvLoader, PasswordEncoderConfig, LoginGateConfig	Start-up setup: the system clock, the sample data loaded from jobs.csv, the password encoder and the identity check.

8.2 Key Design Decisions

- Names come from the design. Every controller, service, entity and public operation takes its name from the design. The name is a lifeline (participant) or message in Section 6, or a class in Section 7. New operations are added next to the existing ones. No one renames another member's classes or operations.
- Rules live in the domain objects. A JobPosting decides whether it is open and matches the search. An Application decides which stage changes are allowed. A UserAccount applies its own status change. Services only coordinate and check rules across objects, such as "the last active administrator cannot be deactivated".
- Time is injected. Every service reads the date and time from a java.time.Clock bean, never from LocalDate.now(). Tests can freeze the clock and check the closing-date rules exactly.
- Errors follow one convention. Business-rule failures return HTTP 400 with a message that names the problem. Unknown identifiers or email addresses return HTTP 404. A wrong email or password returns 401 and a deactivated account 403. With the identity check on, a request that names another user returns 403. A duplicate email returns 400. Four small handler classes map the exceptions to these codes.
- Sample data is a file. The five sample postings are in backend/src/main/resources/jobs.csv. JobCsvLoader imports them at start-up. DevDataSeeder also creates an administrator, a recruiter, an applicant, a deactivated applicant and two applications in the OFFER stage with their history. Every use case can be tried right away.
- Passwords are hashed. Register Account stores a BCrypt hash (a one-way scrambled form of the password). The plain password never reaches an entity or a log.
- Identity comes from the session. User Login (LoginController, AuthenticationService) checks the email and password against the stored BCrypt hash. A deactivated account is refused. The signed-in user is kept in the HTTP session. The endpoints still read the caller from the X-Applicant-Id, X-Recruiter-Id and X-Administrator-Id headers. They also read it from the id parameters and the /api/applicants/{id} paths. IdentityCheckInterceptor, a check that runs before every request, rejects any id that does not belong to the signed-in user. The setting rams.login.enforce turns this check on. It is off in the tests.
- Inheritance as in the domain model. UserAccount is an abstract entity (one database table per class). Applicant, Recruiter and Administrator extend it, so one account has exactly one role. The role is the type of the object. Manage Accounts and Roles changes a status on the object itself. A role change replaces the object. The new Applicant, Recruiter or Administrator keeps the email, password hash, name and status of the old one. The change is refused while the account still owns

applications, resumes, job postings or recorded account changes (extension 6c). A new recruiter also needs a company (extension 6d).

8.3 Subsystems

The classes of Figure 3 form four subsystems, one per actor plus a shared core (Table 3). Each subsystem has its own controllers, services, entities, repositories and endpoints. Subsystems mostly share entities and repositories. One exception: JobSearchController in the Applicant subsystem calls JobPostingService of the Recruiter subsystem. Every subsystem reaches UserAccount through the core. This split let each member work on their own classes. The recruiter uses three endpoints that live in ApplicationController and ApplicationService, because they work on an Application: Update Application Stage, Extend Job Offer and the resume download. IdentityCheckInterceptor and DemoIntegrationController serve every subsystem and belong to none.

Table 3. Subsystems of the back end

Subsystem	Responsibility	Classes	Members
Applicant subsystem	Search open postings; submit, list and track applications with their stage history; view and update the applicant profile	JobSearchController, ApplicationController, ResumeController, ApplicantProfileController, ApplicationService, ApplicantProfileService, Applicant, Application, StageRecord, Resume, ApplicantRepository, ApplicationRepository, ResumeRepository	Luyu Niu (search), Yuting Mei (submit), Spencer Hammack (tracking, stage history, withdrawal), Xinyi Chen (profile)
Recruiter subsystem	Create and publish job postings. Move applications through the stages (Update Application Stage, Extend Job Offer). Keep recruiter and company records.	JobPostingController, JobPostingService, JobPosting, Recruiter, Company, JobPostingRepository, RecruiterRepository, CompanyRepository	Luyu Niu (postings), Peter Chen (update stage), Spencer Hammack (extend offer)
Administrator subsystem	Look up accounts; change role and status	AccountController, AccountService, Administrator, AdministratorRepository	Luyu Niu
Account core (shared)	Register accounts and sign users in; hold the abstract UserAccount that every role extends	RegisterAccountController, RegistrationService, RegistrationValidator, LoginController, AuthenticationService, UserAccount, UserAccountRepository	Xinyi Chen

The Java packages on main are still organized by layer (web, service, domain, repository). That is the Spring Boot convention we started from. The four subsystems are the logical grouping of Figure 3 and Table 3. In Iteration 3, after all branches are merged, we will move the classes into one package per subsystem. Doing it now would cause merge conflicts.

8.4 Technical Details

Platform. The back end runs on Java 21 with Spring Boot 4. Spring Web MVC serves the REST endpoints. Spring Data JPA with Hibernate maps the entities to an H2 database created in memory at start (console at /h2-console). Spring Security Crypto provides BCrypt password hashing. JUnit 5 with Spring Boot Test

runs the tests. The Maven wrapper (mvnw) downloads the build tool, so only a Java 21 JDK (Java Development Kit) and Git are needed.

Request handling. Every endpoint consumes and produces JSON (Table 4). The exceptions are the resume upload and download (files) and the logout (no body). Errors follow the convention of Section 8.2. The caller's identity comes from the X-Applicant-Id, X-Recruiter-Id and X-Administrator-Id headers. Without a sign-in the answer is HTTP 401. A header, parameter or path that names another user gets HTTP 403. The services check that the account is active and owns the record it touches.

Data. DevDataSeeder creates the sample accounts and JobCsvLoader imports backend/src/main/resources/jobs.csv at start, so every run begins with the same data. With the demo setting (Spring profile) the database is kept in a file between restarts. Stages and statuses are Java enumerations (ApplicationStage, PostingStatus, AccountStatus, Role), so an invalid value cannot be stored.

User interface. The React front end in the frontend folder runs on its own development server (npm run dev, port 5173). It forwards every /api request to the back end on port 8080, so no extra browser security setup (CORS) is needed. Its pages read their data through small endpoints served by DemoIntegrationController. Each one returns just what one page shows (/api/demo/actors, jobs, applications, accounts, saved). The pages write through the use case endpoints of Table 4. The job search calls the Search Available Jobs endpoint. The user signs in with User Login. The front end keeps the id of the signed-in user and sends it with every request. The two plain pages submit-application.html and applications.html in src/main/resources/static come from the same Spring Boot process.

Table 4. REST endpoints by use case

Use case	Method and path	Controller
4.2.1.1 Search Available Jobs	GET /api/jobs/search GET /api/jobs/search/results?keyword=&location=&employmentType=	JobSearchController
4.2.1.2 Register Account	POST /api/accounts/register	RegisterAccountController
4.2.1.3 User Login	POST /api/accounts/login (body {email, password}); GET /api/accounts/me; POST /api/accounts/logout	LoginController
4.2.1.4 Update Applicant Profile	GET, PUT /api/profile (applicant from the X-Applicant-Id header; name, phone, location, headline and bio on the Applicant entity)	ApplicantProfileController
4.2.1.5 Submit Job Application	GET /api/applicants GET, POST /api/applicants/{id}/resumes POST /api/applications GET /api/applications?applicantId=	ResumeController, ApplicationController
4.2.1.6 Save Job Posting	GET, PUT /api/demo/saved/{applicantId} (list of saved posting ids)	DemoIntegrationController
4.2.1.7 Track Application Stage	GET /api/applications/tracking (applicant from the X-Applicant-Id header) GET /api/applications/{id} Both require X-Applicant-Id	ApplicationController
4.2.1.8 Withdraw an Application	POST /api/applications/{id}/withdraw (applicant from the X-Applicant-Id header; no request body)	ApplicationController
4.2.2.1 Create and Publish a Job Posting	GET /api/postings/new POST /api/postings POST /api/postings/{id}/publish	JobPostingController

Use case	Method and path	Controller
4.2.2.2 Manage Job Posting	PUT /api/postings/{id} (edit details); POST /api/postings/{id}/close (recruiter from the X-Recruiter-Id header)	JobPostingController
4.2.2.3 Review Submitted Applications	GET /api/demo/applications?recruiterId= (applications to the recruiter's postings, with stage history); GET /api/applications/{id}/resume (resume download, X-Recruiter-Id)	DemoIntegrationController, ApplicationController
4.2.2.4 Update Application Stage	PATCH /api/applications/{id}/stage (recruiter from the X-Recruiter-Id header; body {"stage": ...})	ApplicationController
4.2.2.5 Extend Job Offer	POST /api/applications/{id}/extend-offer (the React page records an offer through the stage endpoint of 4.2.2.4) Requires X-Recruiter-Id; no request body	ApplicationController
4.2.3.1 Manage Accounts and Roles	GET /api/accounts/change GET /api/accounts/lookup?email= PUT /api/accounts/{id}/role-status (body {newRole, newStatus, companyName}; a new recruiter needs the company)	AccountController

9. Testing

Testing is automated with JUnit 5 and runs with ./mvnw test. At the Iteration 2 submission the main branch is Release v2.5.2 (commit 0398ebb). It holds 134 tests in 29 test classes. All pass. The React front end has one unit test (Vitest), and the TypeScript compiler checks its types.

9.1 JUnit Testing Summary

Tests exist at five levels (Table 5): domain, service, web, utility and start-up. The first three match the layers of Section 8.

- Domain unit tests check one object without Spring. They cover search matching, open and closed postings, and the allowed stage changes and withdrawal of an application. Table 5 lists the classes.
- Service tests start the application in test mode with an in-memory H2 database and a fixed clock. They check the business rules and the stored result of each use case, including the extensions.
- Web tests call the controllers through a simulated HTTP client (MockMvc), with the services replaced by fakes. They check the HTTP status and the JSON body, and that no password or hash is ever sent.
- Utility tests (JobCsvParserTest, RegistrationValidatorTest) cover the CSV parser and the registration input rules, including Unicode and length limits.

Table 5. Test classes on the main branch

Test class	Level	Tests	Written by
JobPostingTest	Domain	6	Luyu Niu
ApplicationTest	Domain	6	Peter Chen
ApplicationStageHistoryTest	Domain	3	Spencer Hammack
ApplicationWithdrawalTest	Domain	5	Spencer Hammack
UpdateApplicationStageServiceTest	Service	8	Peter Chen

Test class	Level	Tests	Written by
UpdateApplicationStageIntegrationTest	Service	1	Peter Chen
JobPostingServiceTest	Service	6	Luyu Niu
AccountServiceTest	Service	8	Luyu Niu
ApplicationTrackingServiceTest	Service	5	Spencer Hammack
ApplicationHistoryIntegrationTest	Service	2	Spencer Hammack
ApplicationWithdrawalServiceTest	Service	7	Spencer Hammack
SubmitApplicationIntegrationTest	Service	1	Yuting Mei
RegistrationIntegrationTest	Service	8	Xinyi Chen
JobCsvLoaderIntegrationTest	Service	3	Xinyi Chen
LoginIntegrationTest	Service	7	Xinyi Chen
ApplicantProfileIntegrationTest	Service	4	Xinyi Chen
JobSearchControllerTest	Web	4	Luyu Niu
JobPostingControllerTest	Web	7	Luyu Niu
AccountControllerTest	Web	6	Luyu Niu
ApplicationTrackingControllerTest	Web	3	Spencer Hammack
ApplicationWithdrawalControllerTest	Web	5	Spencer Hammack
RegistrationStartupHttpTest	Web	1	Xinyi Chen
RegisterAccountPayloadTest	Web	2	Xinyi Chen
LoginGateTest	Web	7	Xinyi Chen
UpdateApplicationStageControllerTest	Web	5	Peter Chen
RegistrationValidatorTest	Utility	8	Xinyi Chen
JobCsvParserTest	Utility	4	Xinyi Chen
SystemInfoControllerTest, RecruitingSystemApplicationTests	Start-up	2	Luyu Niu

9.2 Test Cases Table

Table 6 lists the test cases. We chose them with the three black-box techniques of the lecture, which look only at inputs and outputs. EP is equivalence partitioning: one value per valid or invalid partition. BVA is boundary value analysis: the value at and next to a limit. Req is a requirement-based test: the main flow or an extension of a use case. Every implemented use case has at least one test case. A use case that enforces a rule has a second one for that extension. A status marked JUnit comes from the test classes of Table 5. A status marked manual was checked by hand in the React front end on Release v2.5.

Table 6. Test cases

ID	Use case	Tester	Technique	Precondition	Input / action	Expected result	Actual	Status
TC-01	4.2.1.1 Search Available Jobs	Luyu Niu	Req, main flow	Three open postings, one expired posting, one draft	Search with no criteria	The three open postings, newest first	As expected	Pass, JUnit
TC-02	4.2.1.1 Search Available Jobs	Luyu Niu	BVA, ext. 4a	Open postings exist; keyword limit is 100 characters	Keyword of 101 characters	HTTP 400, message names the keyword; no search runs	As expected	Pass, JUnit

ID	Use case	Tester	Technique	Precondition	Input / action	Expected result	Actual	Status
TC-03	4.2.2.1 Create and Publish a Job Posting	Luyu Niu	EP, ext. 4a	Recruiter identified	Title and closing date only	HTTP 400 listing description, location and employment type; nothing saved	As expected	Pass, JUnit
TC-04	4.2.2.1 Create and Publish a Job Posting	Luyu Niu	BVA, ext. 4b	Recruiter identified; closing date must be later than today	Closing date = today	HTTP 400 naming the closing date; nothing saved	As expected	Pass, JUnit
TC-05	4.2.2.1 Create and Publish a Job Posting	Luyu Niu	Req, ext. 5a3	Draft posting whose closing date was yesterday	Publish the draft	HTTP 400 asking for a new closing date; status stays DRAFT	As expected	Pass, JUnit
TC-06	4.2.3.1 Manage Accounts and Roles	Luyu Niu	EP, ext. 3a	No account with the email nobody@example.com	Look up nobody@example.com	HTTP 404 "No account matches"	As expected	Pass, JUnit
TC-07	4.2.3.1 Manage Accounts and Roles	Luyu Niu	EP, ext. 6a	Exactly one active administrator	Deactivate that administrator	HTTP 400; role and status unchanged	As expected	Pass, JUnit
TC-08	4.2.1.2 Register Account	Xinyi Chen	Req, main flow	Email not registered	Full name, valid email, password of 8+ characters, role APPLICANT	HTTP 201; BCrypt hash stored; an Applicant object is created, no company or recruiter	As expected	Pass, JUnit
TC-09	4.2.1.2 Register Account	Xinyi Chen	EP, ext. 5a	existing@example.com is registered	Register Existing@Example.com	HTTP 400 duplicate email; no company created	As expected	Pass, JUnit
TC-10	Sample data (jobs.csv loader)	Xinyi Chen	Req	Sample jobs.csv with quoted commas and escaped quotes	Start the application	Five postings with correct fields	As expected	Pass, JUnit
TC-11	Sample data (jobs.csv loader)	Xinyi Chen	EP	CSV whose row 2 names a recruiter with no account	Start the application	Nothing imported; error names row 2	As expected	Pass, JUnit
TC-12	4.2.1.5 Submit Job Application	Yuting Mei	Req, main flow	Active applicant, open job, own resume	Submit with a cover letter	HTTP 201; stage SUBMITTED; My Applications returns the same record	As expected	Pass, JUnit
TC-13	4.2.2.4 Update Application Stage	Peter Chen	Req, main flow	Active recruiter; application in SUBMITTED	Target stage UNDER_REVIEW	HTTP 200; stage UNDER_REVIEW; update time recorded	As expected	Pass, JUnit
TC-14	4.2.2.4 Update Application Stage	Peter Chen	EP, invalid input	Active recruiter	Request body {} (no stage)	HTTP 400; stage unchanged	As expected	Pass, JUnit
TC-15	4.2.1.7 Track Application Stage	Spencer Hammack	Req, main flow	Applicant with two applications	List my applications	Two summaries with their stages	As expected	Pass, JUnit
TC-16	4.2.1.7 Track Application Stage	Spencer Hammack	EP	Application belongs to another applicant	View that application	HTTP 400 "own application"	As expected	Pass, JUnit

ID	Use case	Tester	Technique	Precondition	Input / action	Expected result	Actual	Status
TC-17	4.2.1.5 Submit Job Application	Yuting Mei	BVA	Active applicant, valid resume; published job whose closing date has passed	Submit an application	HTTP 400; no application created	As expected	Pass, manual
TC-18	4.2.1.5 Submit Job Application	Yuting Mei	EP	Active applicant, valid resume; closed job with a future closing date	Submit an application	HTTP 400; no application created	As expected	Pass, manual
TC-19	4.2.1.8 Withdraw an Application	Spencer Hammack	Req, main flow	Application in SUBMITTED	Applicant withdraws it	HTTP 200; stage WITHDRAWN; stage record with time and applicant added	As expected	Pass, JUnit
TC-20	4.2.1.8 Withdraw an Application	Spencer Hammack	EP, ext. 4a	Application in OFFER	Applicant withdraws it	HTTP 400; stage stays OFFER	As expected	Pass, JUnit
TC-21	4.2.2.5 Extend Job Offer	Spencer Hammack	Req, main flow	Application in INTERVIEW; recruiter of the posting's company	Extend an offer	HTTP 200; stage OFFER; stage record added	As expected	Pass, manual
TC-22	4.2.2.5 Extend Job Offer	Spencer Hammack	EP, ext. 7a	Application in SUBMITTED	Extend an offer	HTTP 400; stage unchanged (review comes first)	As expected	Pass, manual
TC-23	4.2.2.2 Manage Job Posting	Yuting Mei	Req, main flow	Published posting of the recruiter's company	Close the posting	HTTP 200; status CLOSED; posting no longer in search results	As expected	Pass, manual
TC-24	4.2.2.2 Manage Job Posting	Yuting Mei	EP, special req. 3	Closed posting	Edit its title	HTTP 400 "Closed jobs cannot be edited"; details unchanged	As expected	Pass, manual
TC-25	4.2.2.3 Review Submitted Applications	Peter Chen	Req, main flow	Two applications to the recruiter's company, one to another company	Open Candidates	Only the two applications of the company, with applicant, posting and stage	As expected	Pass, manual
TC-26	4.2.1.6 Save Job Posting	Yuting Mei	Req, main flow	One posting saved	Click Saved on that posting, reload the page	Saved jobs list is empty; the server no longer stores the id	As expected	Pass, manual
TC-27	4.2.1.4 Update Applicant Profile	Xinyi Chen	Req, main flow	Active applicant Ann Lee, phone 254-555-0100	New name, phone and headline with surrounding spaces	HTTP 200; the trimmed values are stored on the applicant and returned	As expected	Pass, JUnit
TC-28	4.2.1.2 Register Account	Xinyi Chen	BVA	Email not registered; minimum password length is 8	Password of 7 characters	HTTP 400 "at least 8"; no account created	As expected	Pass, JUnit
TC-29	4.2.1.3 User Login	Xinyi Chen	Req, main flow	Active applicant registered with password P	Sign in with the email and P	HTTP 200; the session reports the applicant id and role; the response holds no password	As expected	Pass, JUnit
TC-30	4.2.1.3 User Login	Xinyi Chen	EP, ext. 5a	Registered email	Sign in with a wrong password	HTTP 401 with the same message as for an unknown email; no session created	As expected	Pass, JUnit
TC-31	4.2.1.3 User Login	Xinyi Chen	EP, ext. 5b	Deactivated account with the right password	Sign in	HTTP 403 reporting that the account is inactive	As expected	Pass, JUnit

ID	Use case	Tester	Technique	Precondition	Input / action	Expected result	Actual	Status
TC-32	4.2.3.1 Manage Accounts and Roles	Luyu Niu	Req, main flow	Applicant Ann with no applications or resumes; administrator Ada	New role ADMINISTRATOR	HTTP 200; the account is now an Administrator object with the same email and name; the old object is gone; change recorded with Ada and the time	As expected	Pass, JUnit
TC-33	4.2.3.1 Manage Accounts and Roles	Luyu Niu	EP, ext. 6c	Applicant who owns one resume	New role ADMINISTRATOR	HTTP 400 naming the resumes; the account stays an Applicant	As expected	Pass, JUnit
TC-34	4.2.3.1 Manage Accounts and Roles	Luyu Niu	EP, ext. 6d	Applicant	New role RECRUITER without a company name	HTTP 400 asking for the company; account unchanged	As expected	Pass, JUnit

10.Team Contribution

Table 7 shows what each member produced in Iteration 1. Table 8 shows Iteration 2.

Table 7. Contribution in Iteration 1

Member	Role	Contribution in Iteration 1
Xinyi Chen	Requirements	Analyzed the functional and non-functional requirements. Wrote the use cases and sequence diagrams for Register Account, User Login and Update Applicant Profile.
Yuting Mei	Design	Wrote three use cases and their sequence diagrams: Submit Job Application and Save Job Posting (applicant) and Manage Job Posting (recruiter). Drew UI sketches and low-fidelity wireframes.
Spencer Hammack	Librarian	Owned Track Application Stage, Withdraw an Application, and Extend Job Offer (formerly Schedule an Interview). Created the corresponding use case descriptions and sequence diagrams.
Zhengshi (Peter) Chen	Quality Assurance	Wrote the use cases Update Application Stage and Review Submitted Applications, with descriptions and sequence diagrams. Also wrote Respond to Job Offer, which was removed after the Iteration 1 review.
Luyu Niu	Project Manager	Coordinated the team and led the consistency review across requirements, use cases, sequence diagrams and the domain model. Wrote the introduction and project overview, and built the domain model and the system context figure. Wrote three use cases and their sequence diagrams. Created and maintained the Jira board.

Table 8. Contribution in Iteration 2

Member	Contribution in Iteration 2
Xinyi Chen	Implemented Register Account (RegistrationService, RegistrationValidator, RegisterAccountController, BCrypt hashing) and the sample job file jobs.csv with its parser and loader. Rewrote the development seeder. Implemented User Login (LoginController, AuthenticationService, session, identity check on every request) and Update Applicant Profile (ApplicantProfileController, ApplicantProfileService). Switched the account classes to inheritance from the abstract UserAccount, as in the domain model. Wrote 44 tests.
Yuting Mei	Implemented all of Submit Job Application (resume upload, validation, persistence, My Applications) with the two pages submit-application.html and applications.html. Designed the high-fidelity user interface for all applicant, recruiter and administrator screens. Built the React front end (job search, registration, my applications with stage history) connected to the back end.
Spencer Hammack	Implemented Track Application Stage: the list and details endpoints and the service rules. Added the StageRecord stage history shown in My applications. Implemented Withdraw an Application and Extend Job Offer. Wrote 30 tests. Contributed to the system DCD (Section 7).
Zhengshi (Peter) Chen	Implemented ApplicationService, ApplicationController and Update Application Stage. Wrote 20 tests. Merged Respond to Job Offer into Update Application Stage. Applicants now tell recruiters their decision by email, and only recruiters update the stage. Connected the recruiter and administrator pages of the React front end to the back end (candidates, postings, accounts). Added resume download and the edit and close actions for postings. Maintained the project website.
Luyu Niu	Implemented Search Available Jobs, Create and Publish a Job Posting, and Manage Accounts and Roles. Built the project skeleton (clock, seeder, error handling). Wrote 39 tests. Drew the system DCD (Section 7). Revised the document after the Iteration 1 review. Reviewed, tested and merged all pull requests and published releases v1.0 to v2.5. Maintained the Jira board.

11.Setup / Installation Guide

The back end runs on any computer with a Java 21 JDK (Java Development Kit) and Git. You do not need to install Maven. The repository ships the Maven wrapper (mvnw / mvnw.cmd). It downloads Maven on first use. The React front end also needs Node.js 20 or newer (npm).

11.1 Install and run

- Install a Java 21 JDK (for example Eclipse Temurin 21) and check it with java -version.
- Clone the repository: git clone https://github.com/baylor-csi5324-team3/recruiting-system.git
- Open a terminal in the backend folder and start the application: ./mvnw spring-boot:run (Windows: .\mvnw.cmd spring-boot:run). The first run downloads the dependencies.
- When the log shows “Development data ready”, the application is listening on http://localhost:8080.
- The two plain pages do not sign in. To use them, start the back end with ./mvnw spring-boot:run -Dspring-boot.run.arguments=--rams.login.enforce=false. Then open http://localhost:8080/submit-application.html for the Submit Job Application page and http://localhost:8080/applications.html for My Applications.
- You can inspect the in-memory database at http://localhost:8080/h2-console (connection string jdbc:h2:mem:rams, user sa, no password).
- Front end: open a second terminal in the frontend folder. Run npm install once, then npm run dev. Open http://localhost:5173. Click Sign in at the top right. Use one of the sample accounts of Section 11.3. The password is Demo-pass-2026.

11.2 Run the tests

- In the backend folder run ./mvnw test (Windows: .\mvnw.cmd test). The build prints the number of tests run and ends with BUILD SUCCESS.

11.3 Sample data

Every start loads the same sample data. All accounts share one id sequence. Administrator admin@example.com has id 1. Recruiter recruiter@example.com of company Baylor Tech has id 2. Applicant applicant@example.com (Peter Chen) has id 3 and two applications in the OFFER stage. The deactivated applicant dormant@example.com has id 4. The five postings of jobs.csv are three open, one expired and one draft. The four sample accounts share the development password Demo-pass-2026 (setting rams.seed.demo-password). To keep data between restarts, start with ./mvnw spring-boot:run -Dspring-boot.run.profiles=demo. This keeps the database in a file in backend/data.

12.User Manual

This manual describes what each kind of user can do in the current version (Release v2.5.2). All functions are available in the React front end at http://localhost:5173. Section 11 explains how to start the back end and the front end. Click the Sign in button at the top right. The sample accounts are applicant@example.com (Peter Chen), recruiter@example.com (Rhea Recruiter) and admin@example.com (Ada Admin). The password is Demo-pass-2026. The deactivated account dormant@example.com is refused. The front end sends the id of the signed-in user with every request. The back end checks it against the session. Sign out in the account menu ends the session. The screenshots were taken on October 26, 2026 with the sample data. Release v2.5 replaced the identity menu at the top right with the Sign in button. The pages are unchanged.

12.1 Applicant

- Search available jobs. Sign in as the applicant, or stay a guest. Searching needs no account. The Explore jobs page lists every open posting, newest first (Figure 4). Type a job title keyword in the search box. You can also choose a location or an employment type. Then click Search jobs. The list shows only the matching postings (Figure 5). Clear filters shows all open postings again.
- View a posting and apply. Click View job on a posting (Figure 5) to open its details, then click Apply now (Figure 6). Choose a PDF or DOCX resume file to upload. A cover letter is optional. Click Review application to check the details, then click Submit application (Figure 7). The system confirms the submission and sets the stage to Submitted. You can see the application on the My applications page and track its stage there. You can apply to a posting only once. After you submit, the button changes to Already applied (Figure 8). You cannot apply to a closed or expired posting.
- Save a posting. Click the Save job bookmark on a posting to keep it on the Saved jobs page (Figure 9). Click the bookmark again to remove it. The saved list is stored on the server, so it is still there after you reload the page.
- Track your applications. The My applications page lists every application with its job title, company and current stage (Figure 10). Click View application to see the stage history with the date of every change (Figure 11).
- Withdraw an application. On the application details (Figure 11) click Withdraw and confirm. This works while the application is in the Submitted, Under review or Interview stage. It is not possible after an offer or a final stage.
- Update your profile. The My profile page shows your name, email, phone, location, headline and bio (Figure 12). Change the fields and click Save profile. The back end stores them on your applicant record. The page then shows the saved values. A missing name or an invalid phone number is refused.
- Register. Click Sign in and then Create an account (Figure 13). Enter your full name, an email address and a password of at least eight characters. Choose Applicant or Recruiter. A recruiter also enters the company name. Click Create account. An email address that is already registered is refused. Then sign in with the new email address and password. A deactivated account or a wrong password is refused.

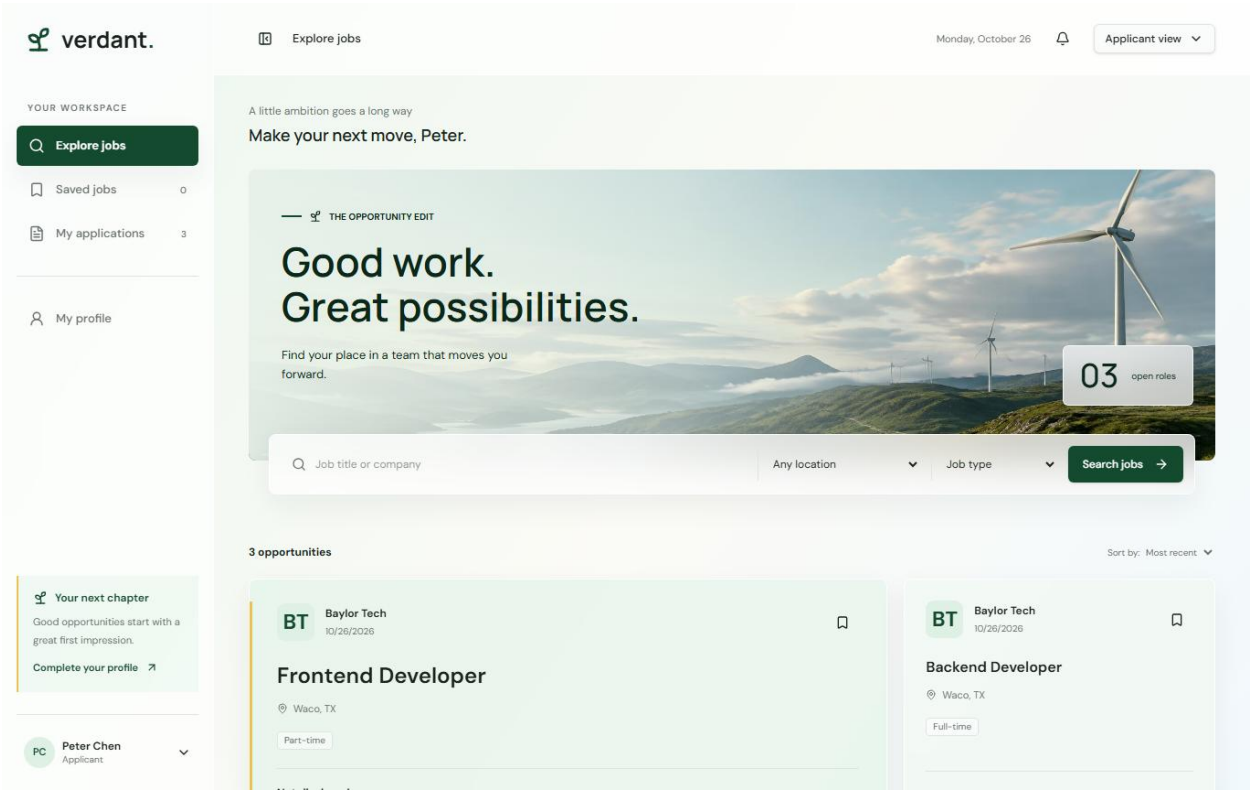


Figure 4. Explore jobs: open postings loaded from the back end, with the search box and filters

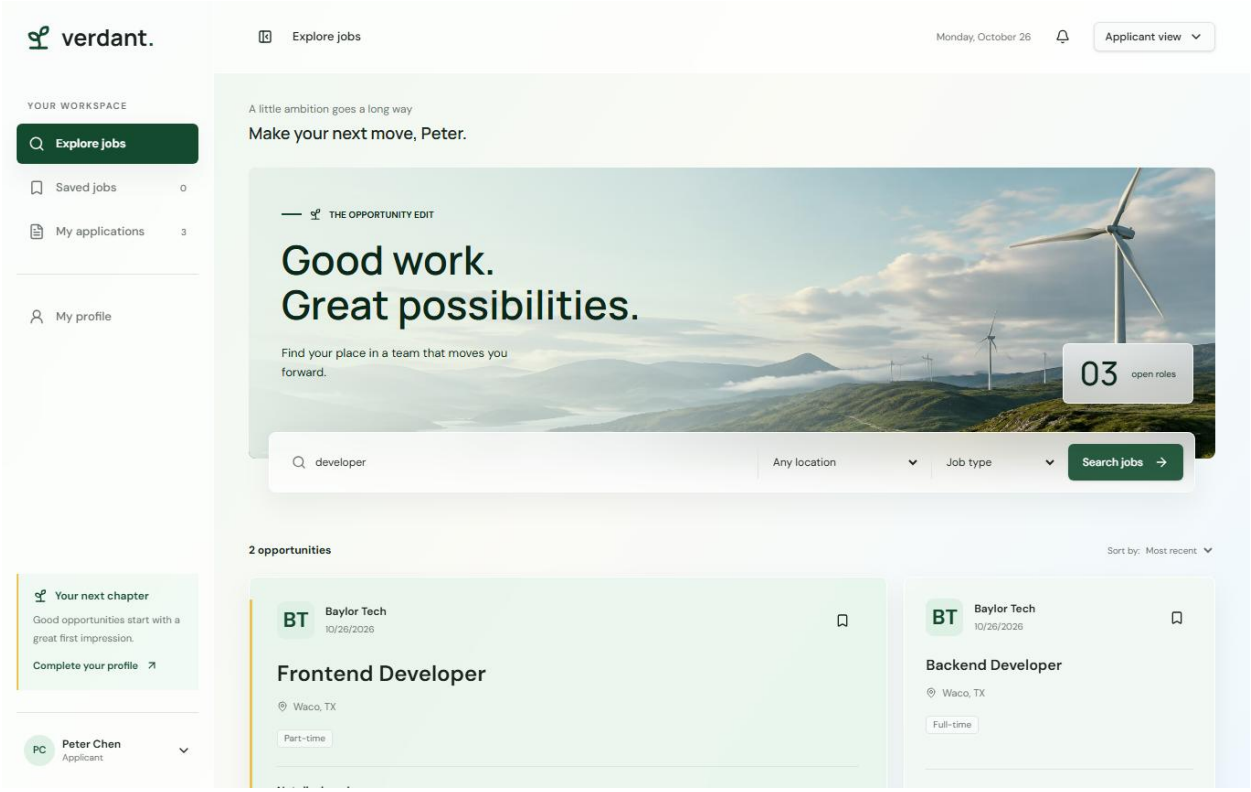


Figure 5. Explore jobs after a keyword search

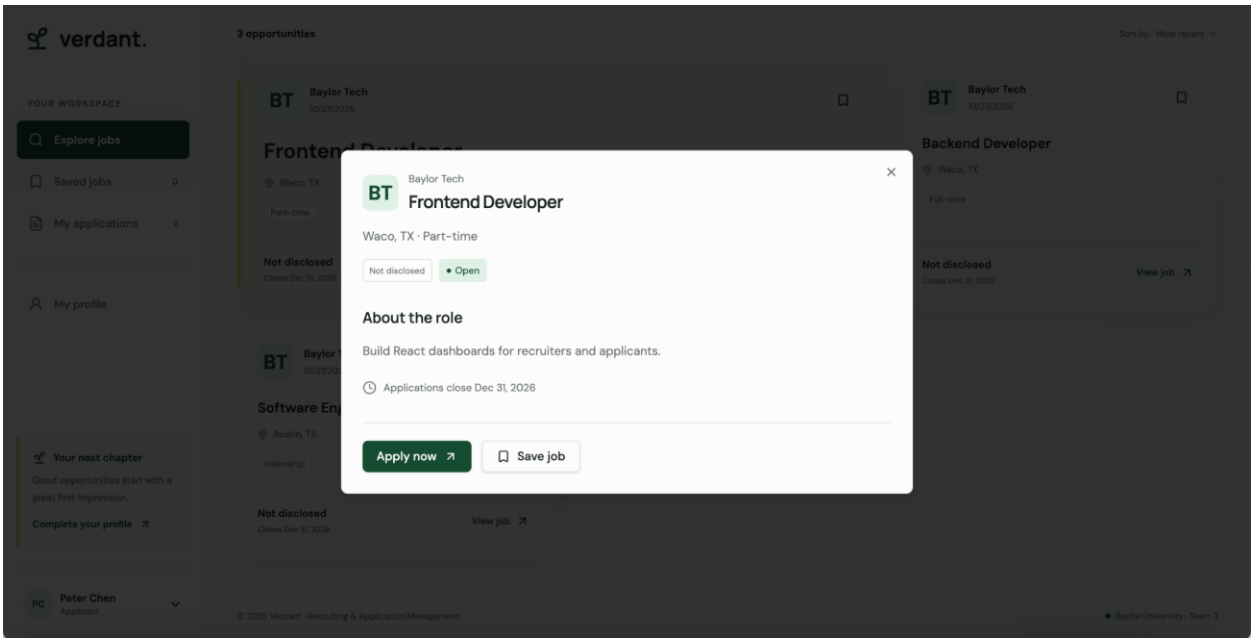


Figure 6. Review job details and apply now

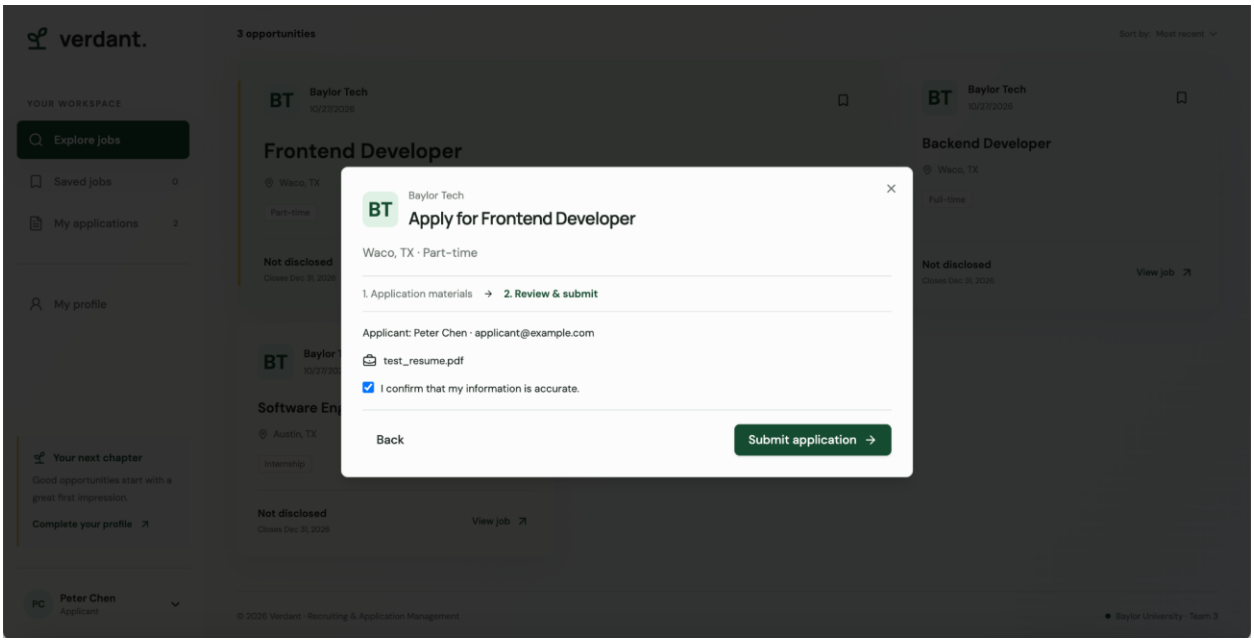


Figure 7. Review and submit application

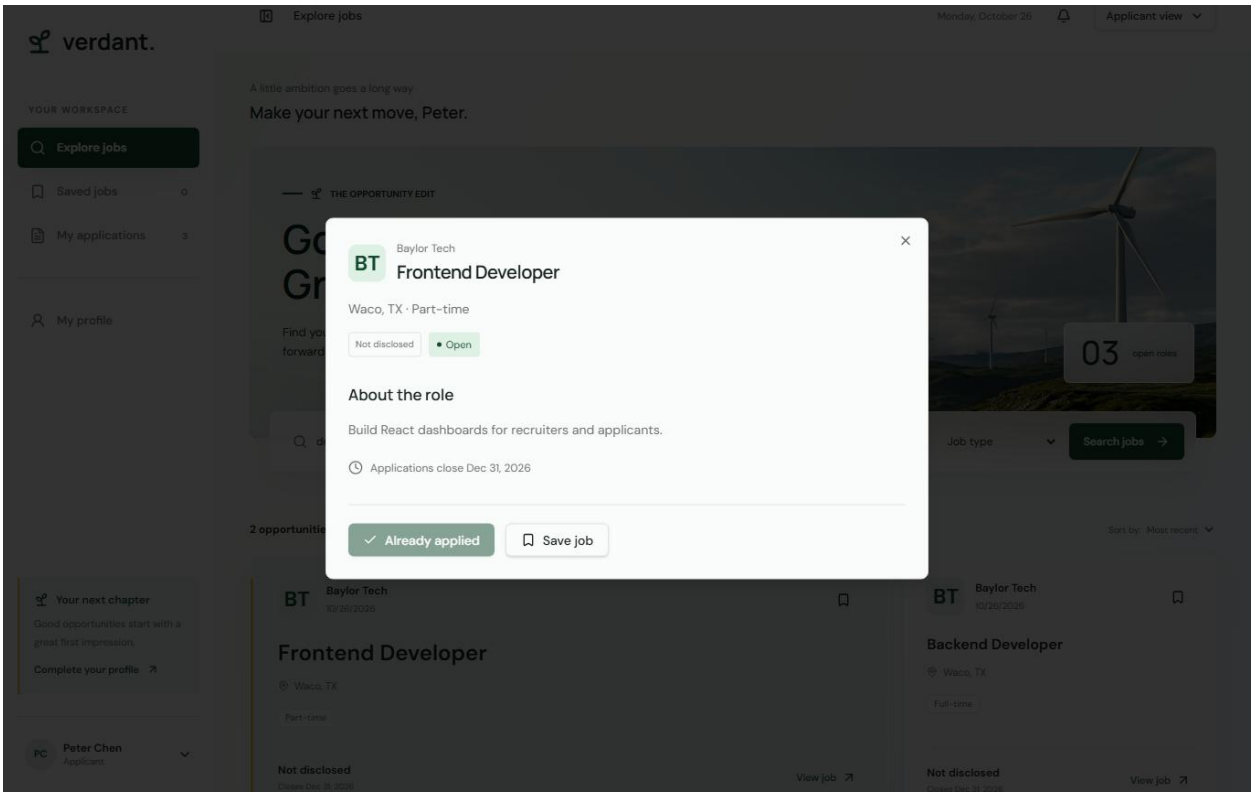


Figure 8. Job details after submission (Already applied)

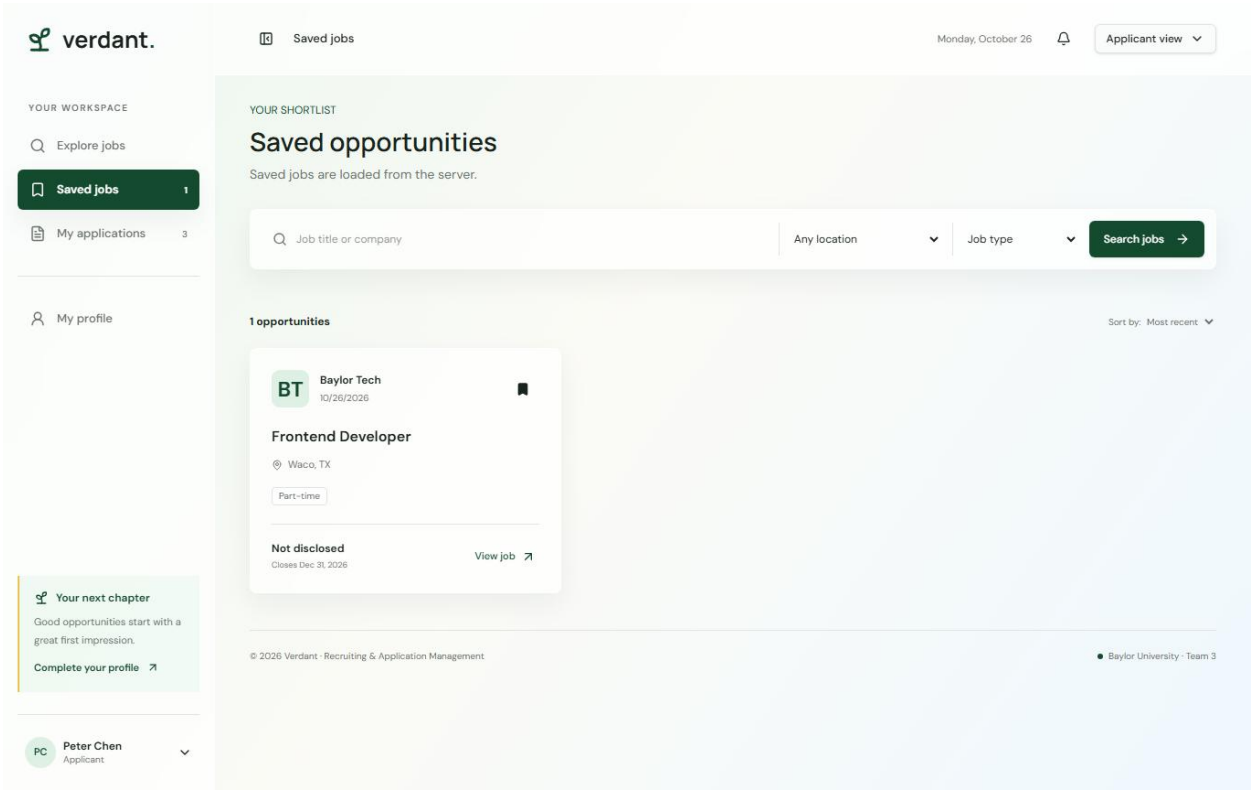


Figure 9. Saved jobs

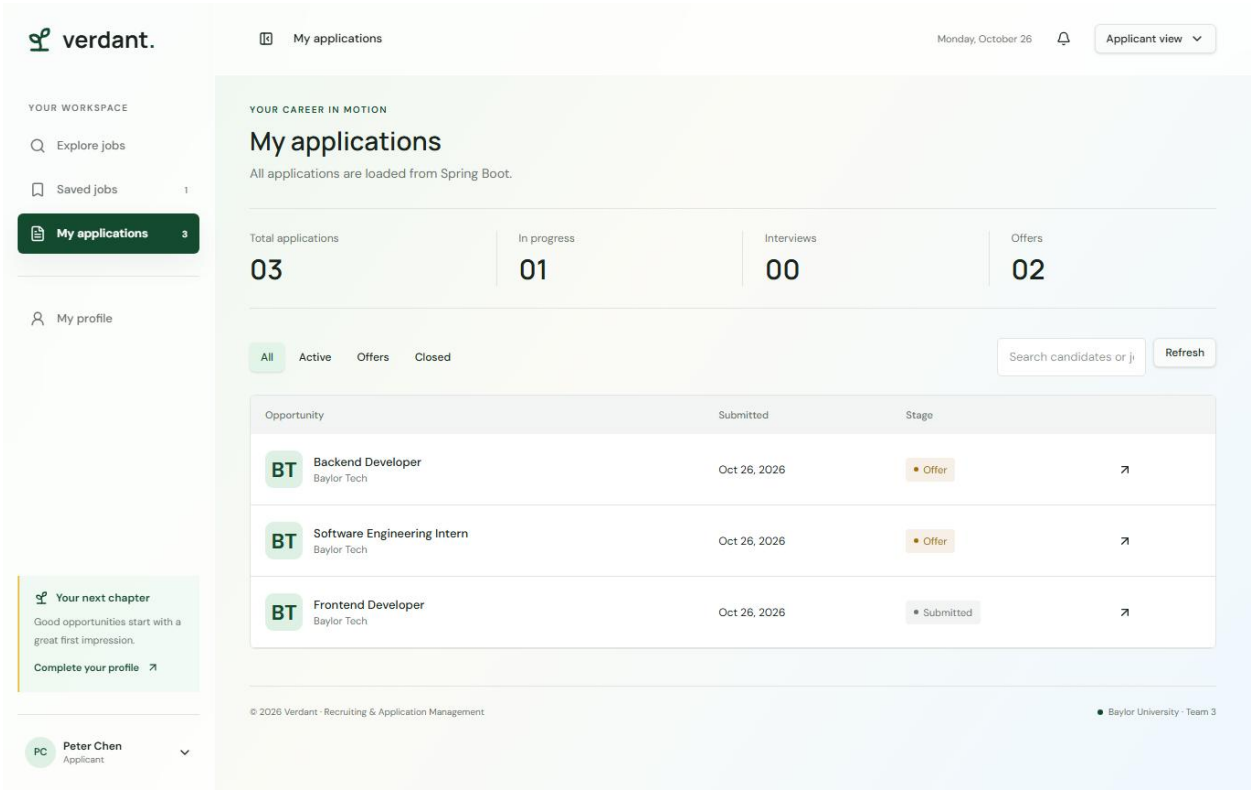


Figure 10. My applications with the current stage of each application

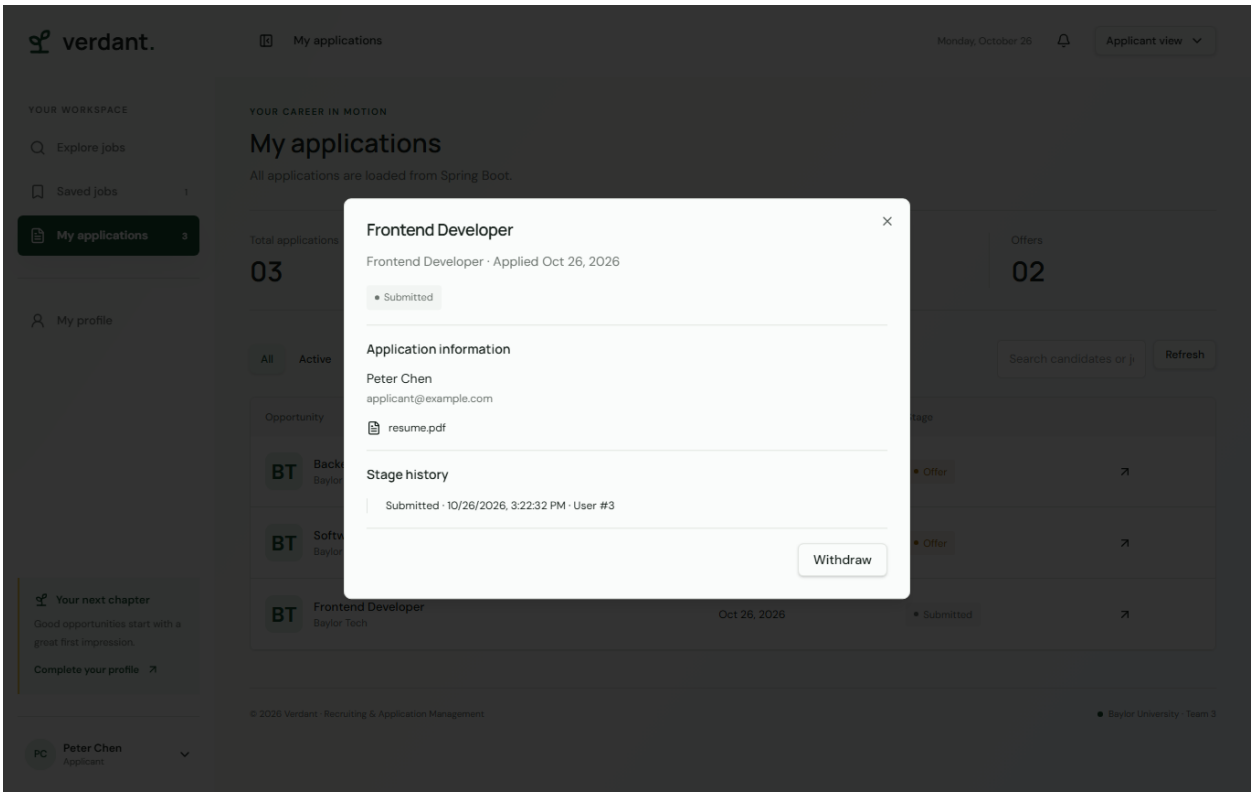


Figure 11. Application details with the stage history and the Withdraw action

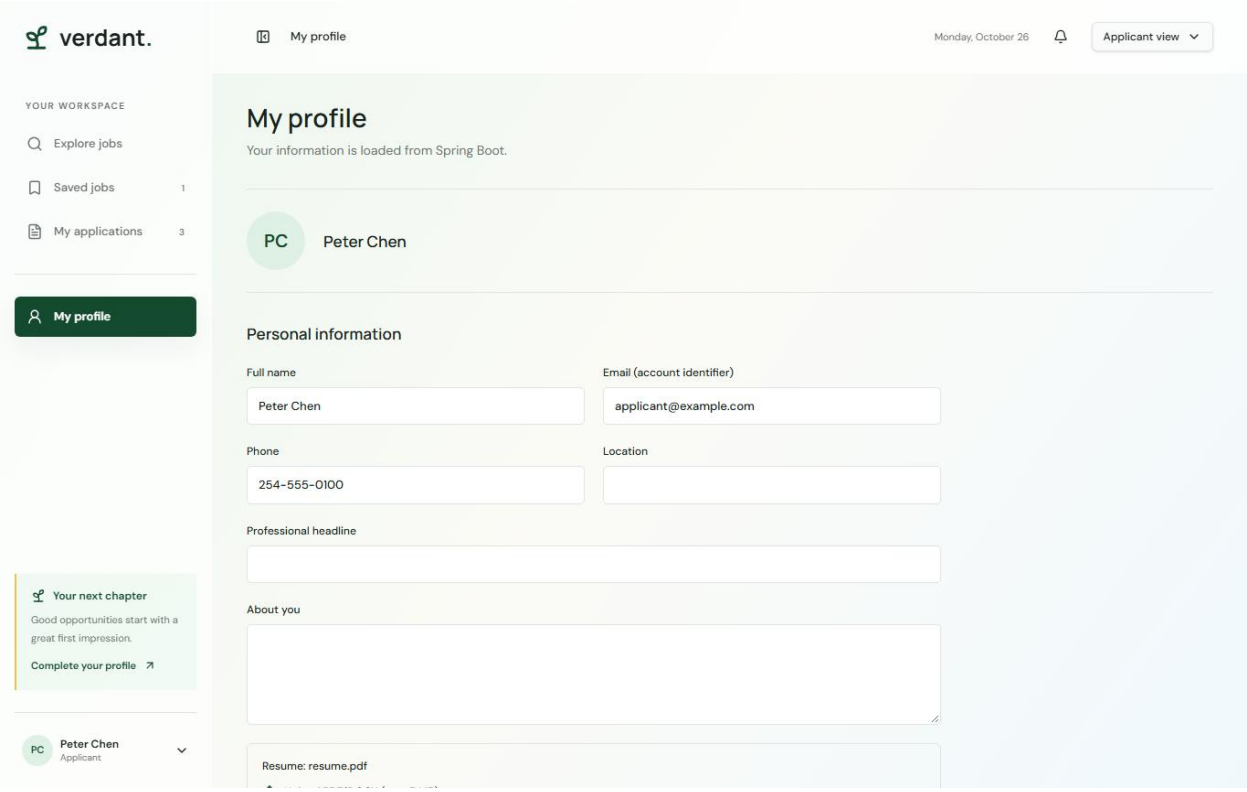


Figure 12. My profile

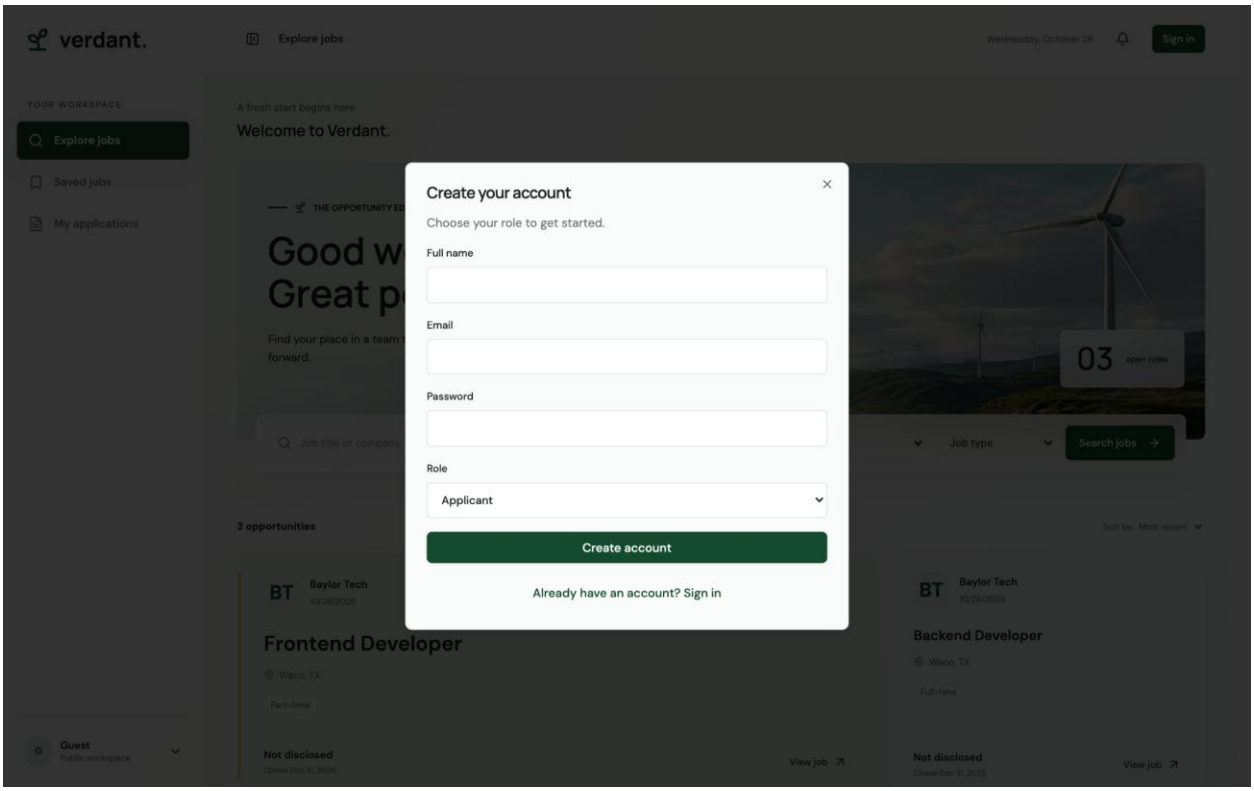


Figure 13. Create an account

12.2 Recruiter

- Create and publish a job posting. Sign in as the recruiter. The Job postings page lists the open, draft and closed postings of your company (Figure 14). Click Create posting (Figure 15). Fill in the title, description, location and employment type. The closing date must be later than today. Save draft keeps the posting invisible to applicants. Publish posting makes it appear in searches until its closing date. A draft can be published later from the list.
- Edit or close a posting. On the Job postings page click Edit to change a draft or an open posting. Click Close posting to stop receiving applications. The system asks for confirmation before closing. A closed posting cannot be reopened.
- Review submitted applications. The Candidates page lists every application to your company's postings (Figure 16). Each row shows the applicant, the posting and the current stage. Click View application to see the stage history and the resume (Figure 17). You can download the resume.
- Update an application stage. The application details page (Figure 17) has stage actions. They are Start review, Move to interview, Extend offer and Reject. After an offer, Record accepted and Record declined appear. The page shows only the actions allowed from the current stage. Every change is kept in the stage history with the date and the user. An accepted, declined, rejected or withdrawn application cannot be changed any more.

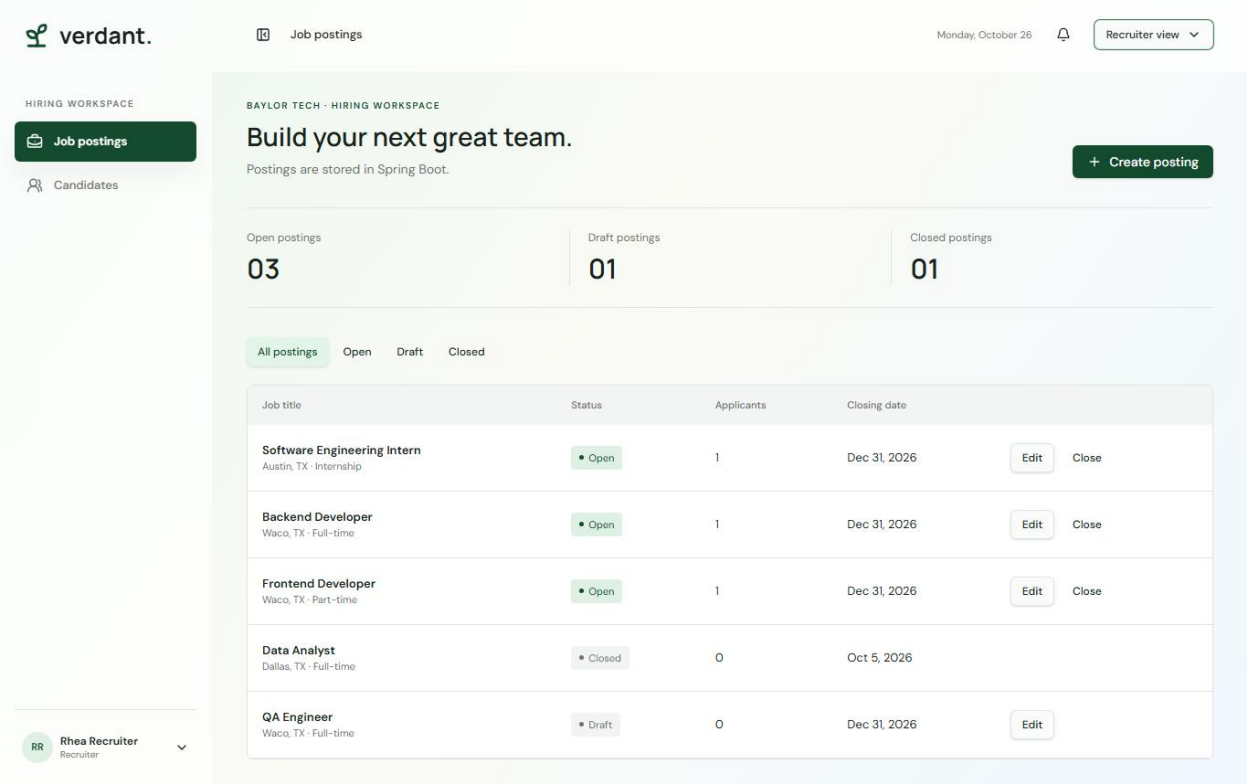


Figure 14. Job postings of the recruiter with the Edit and Close posting actions

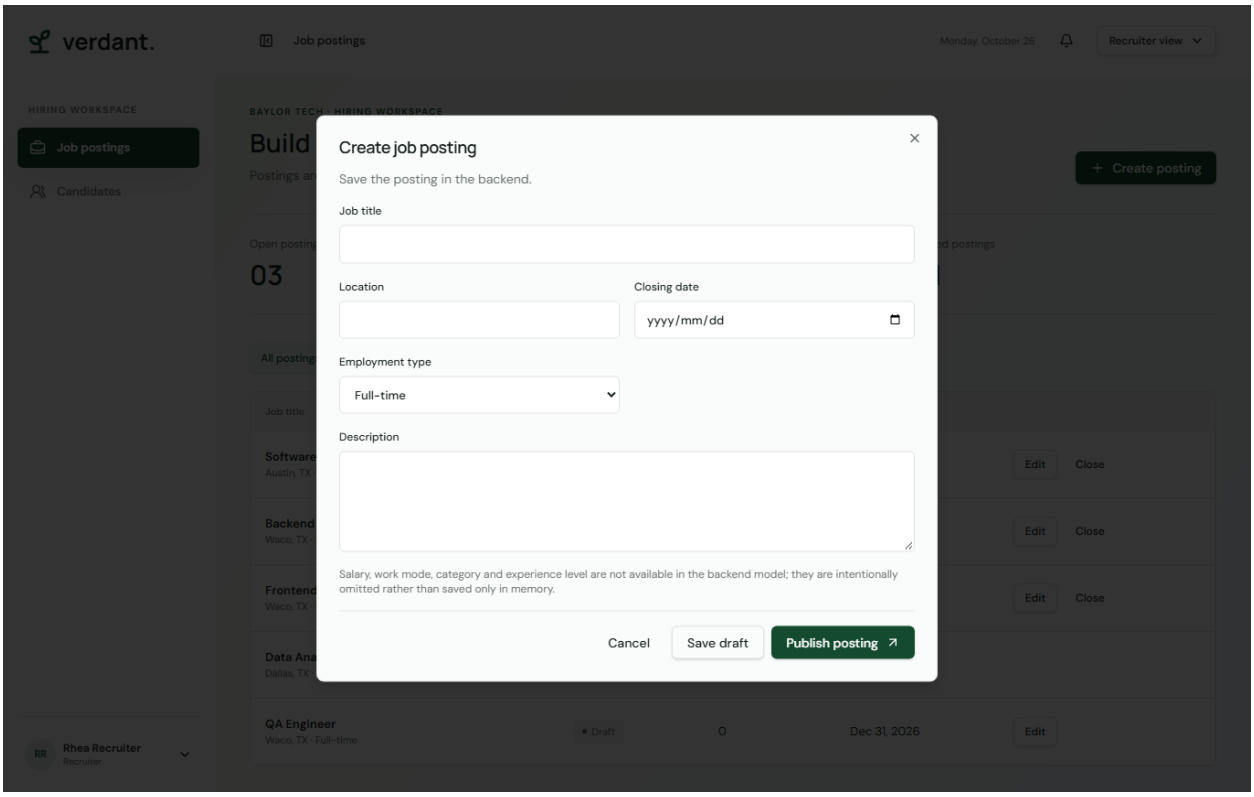


Figure 15. Create job posting form (Save draft or Publish posting)

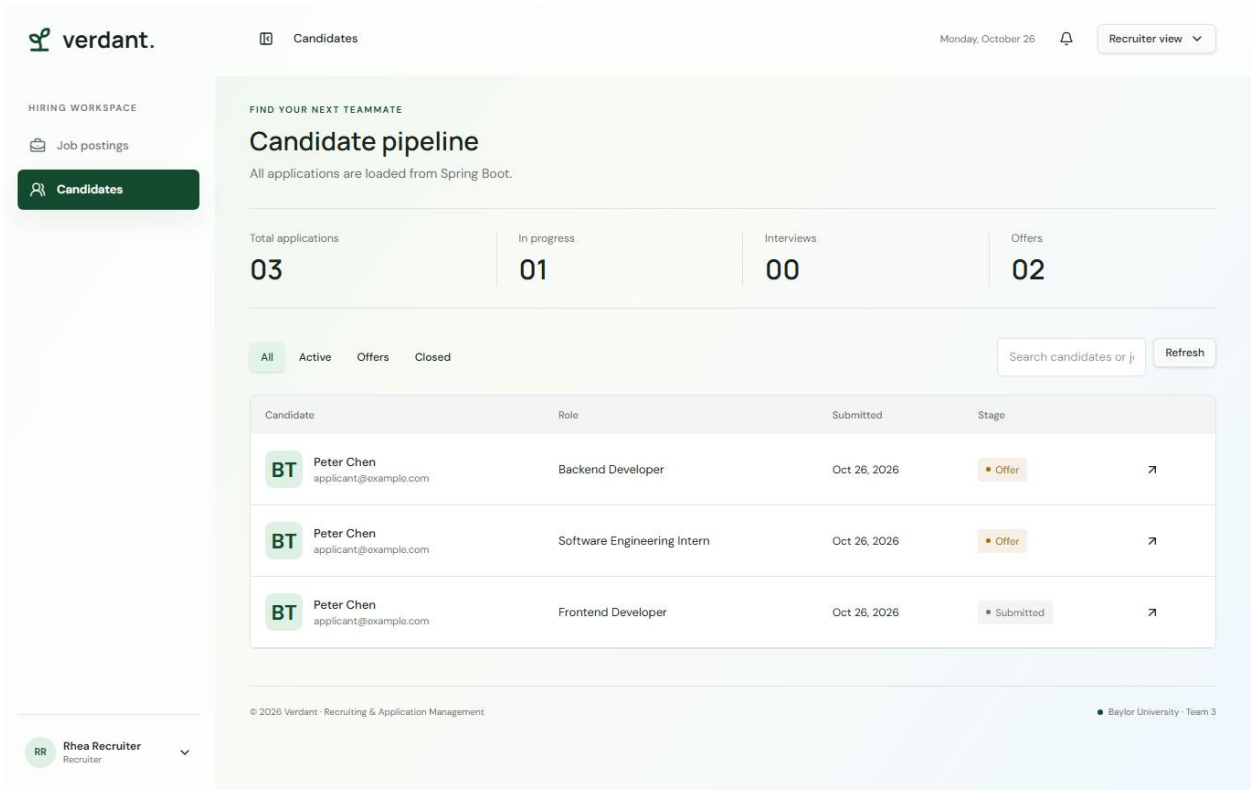


Figure 16. Candidates: applications to the postings of the company

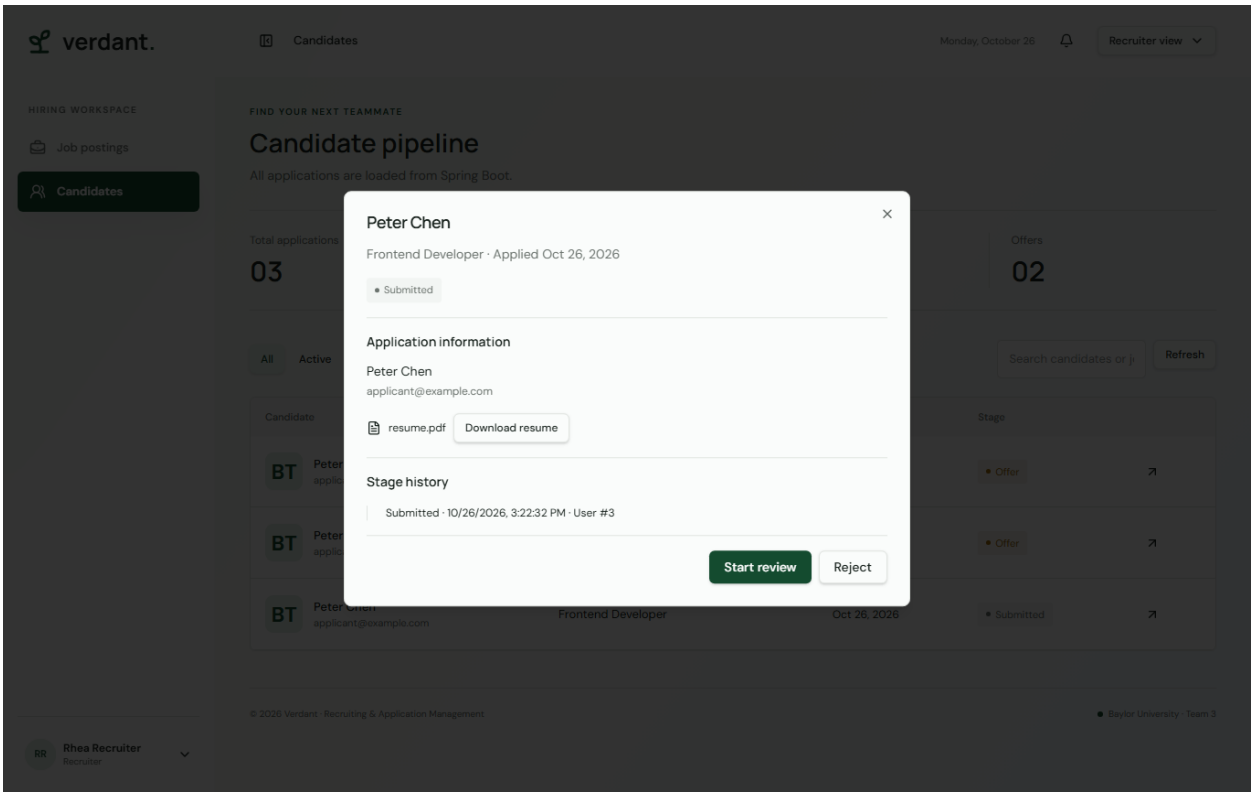


Figure 17. Application details for the recruiter: stage history, resume download and stage actions

12.3 Administrator

- Manage accounts and roles. Sign in as the administrator. The User accounts page lists every account with its email address, role and status (Figure 18). Type a name or email address in the search box to find an account. Change the role (a new recruiter also needs the company name), or choose Activate or Deactivate, and click Confirm change. A role change makes a new account of the new role. It keeps the same email, name and status. It is refused while the account still has applications, resumes or job postings, or has made account changes as an administrator. The system refuses a change that would leave no active administrator. It also reports when the account is already in the requested status.

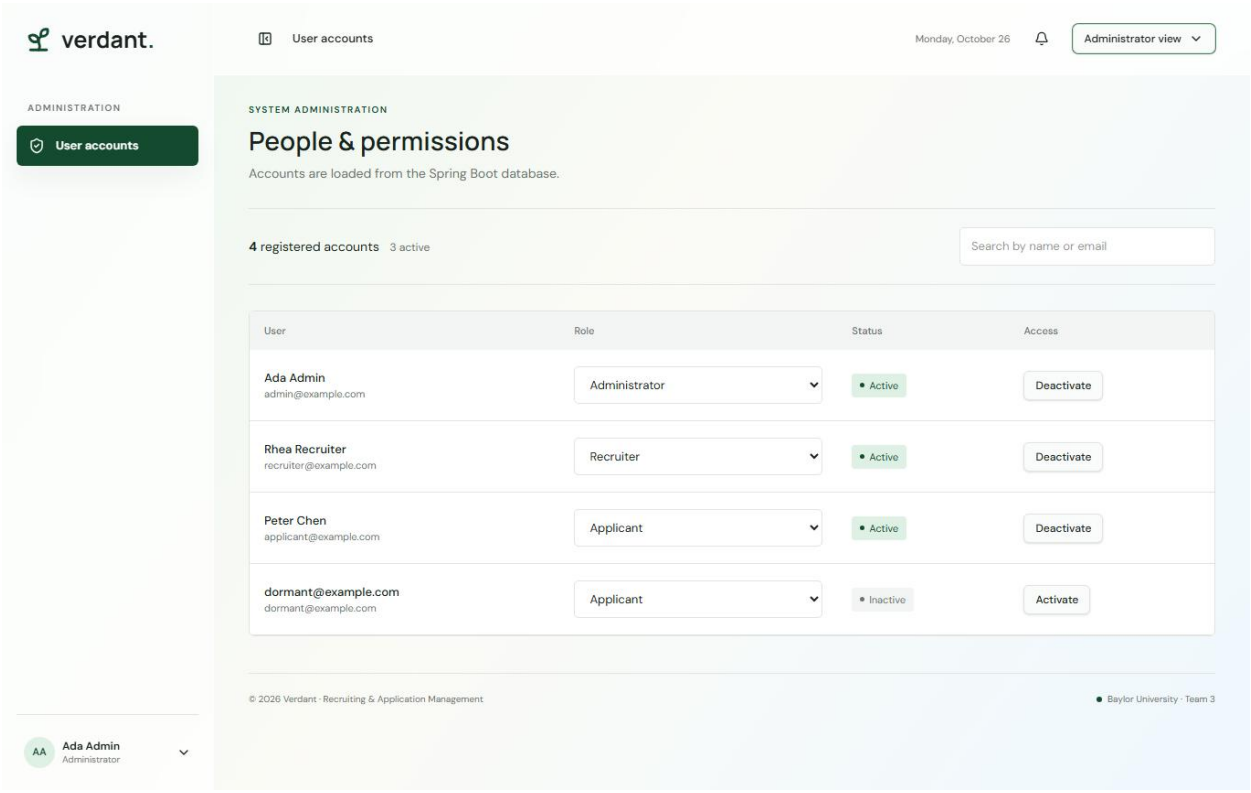


Figure 18. Accounts page of the administrator with the role and status change

12.4 Pages served by the back end

- Two plain HTML pages are also served by the back end at http://localhost:8080. They do not sign in, so start the back end with rams.login.enforce=false (Section 11.1). submit-application.html submits an application (Figure 19). Choose your name and an open posting. Select or upload a resume and add an optional cover letter. Click Review application, then Confirm & Submit. applications.html lists the applications of the chosen applicant with their stage history. Refresh from server reloads them (Figure 20).

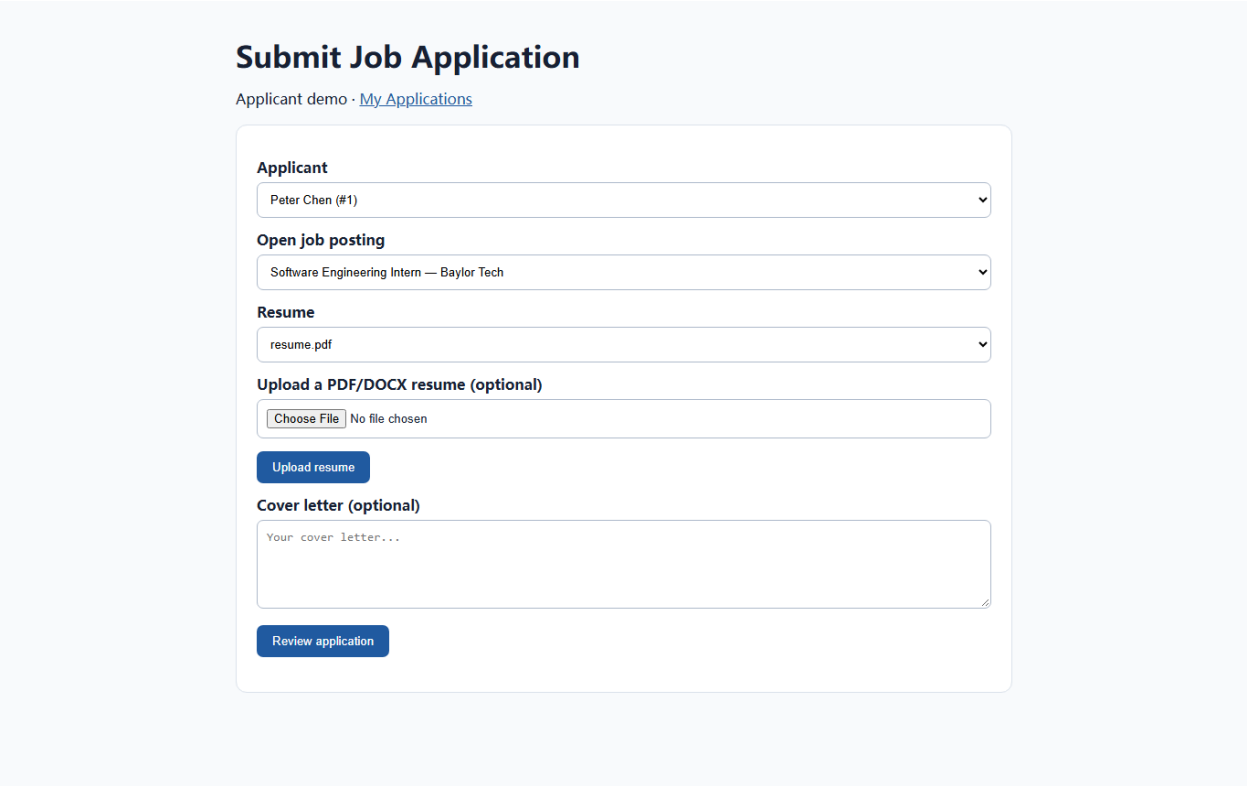


Figure 19. Submit Job Application page served by the back end

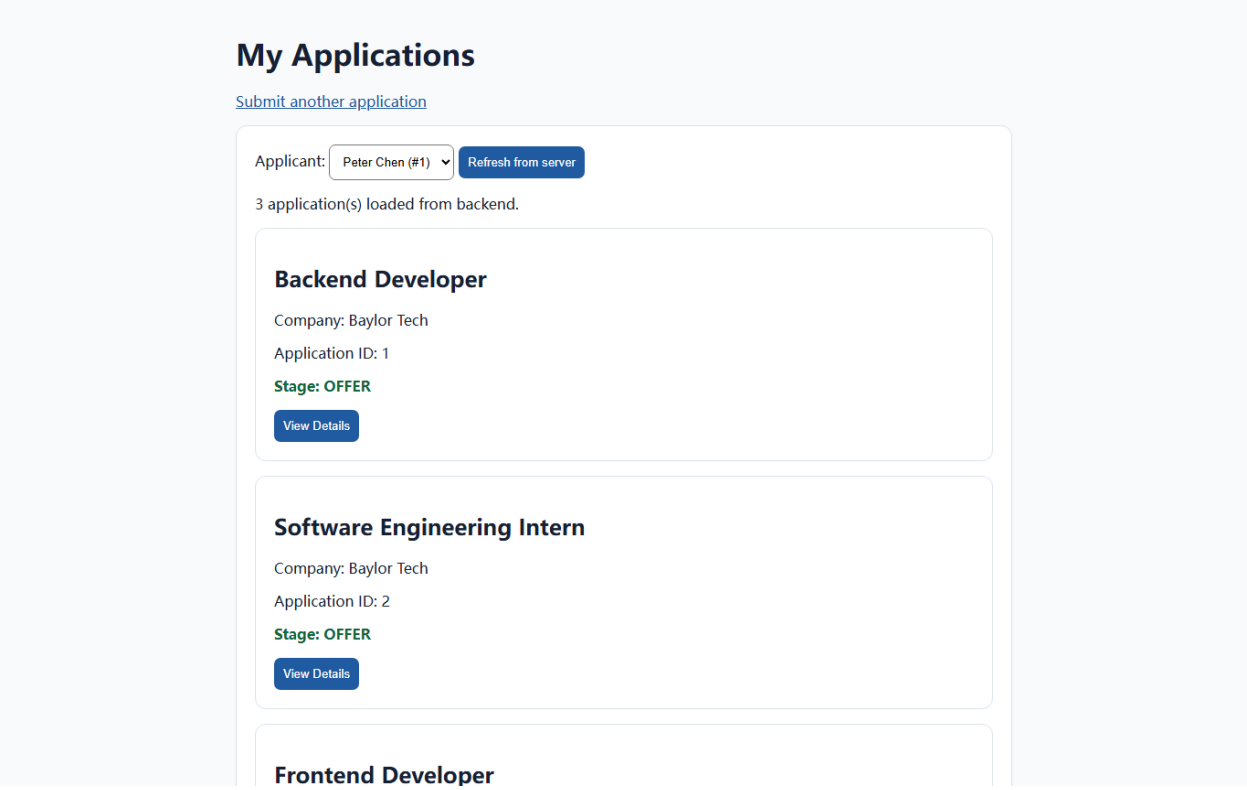
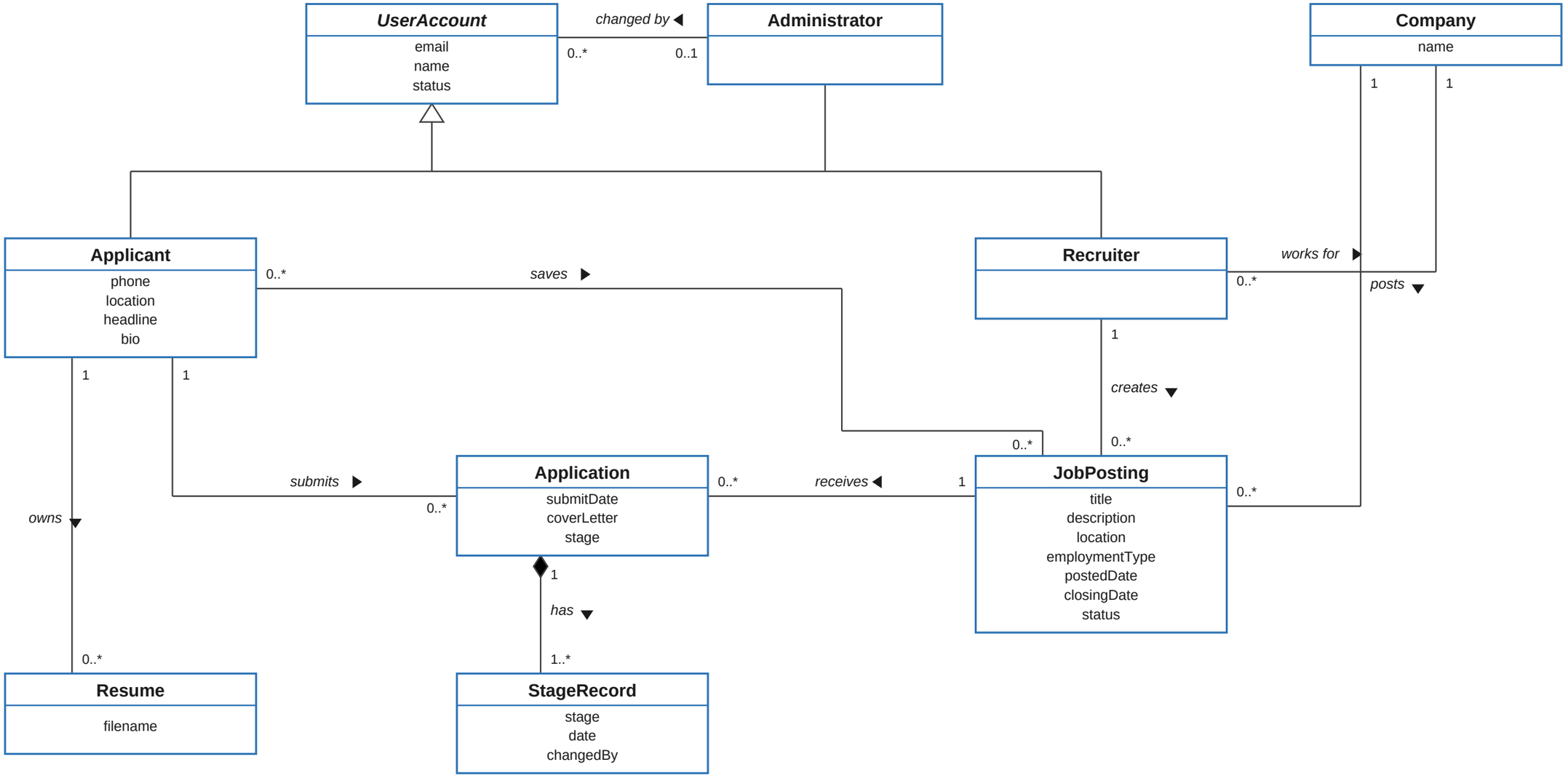


Figure 20. My Applications page served by the back end, with the stage history

Appendix A. Figure 2, domain model, full size (vector; zoom in)



Appendix B. Figure 3, system design class diagram, full size (vector; zoom in)

